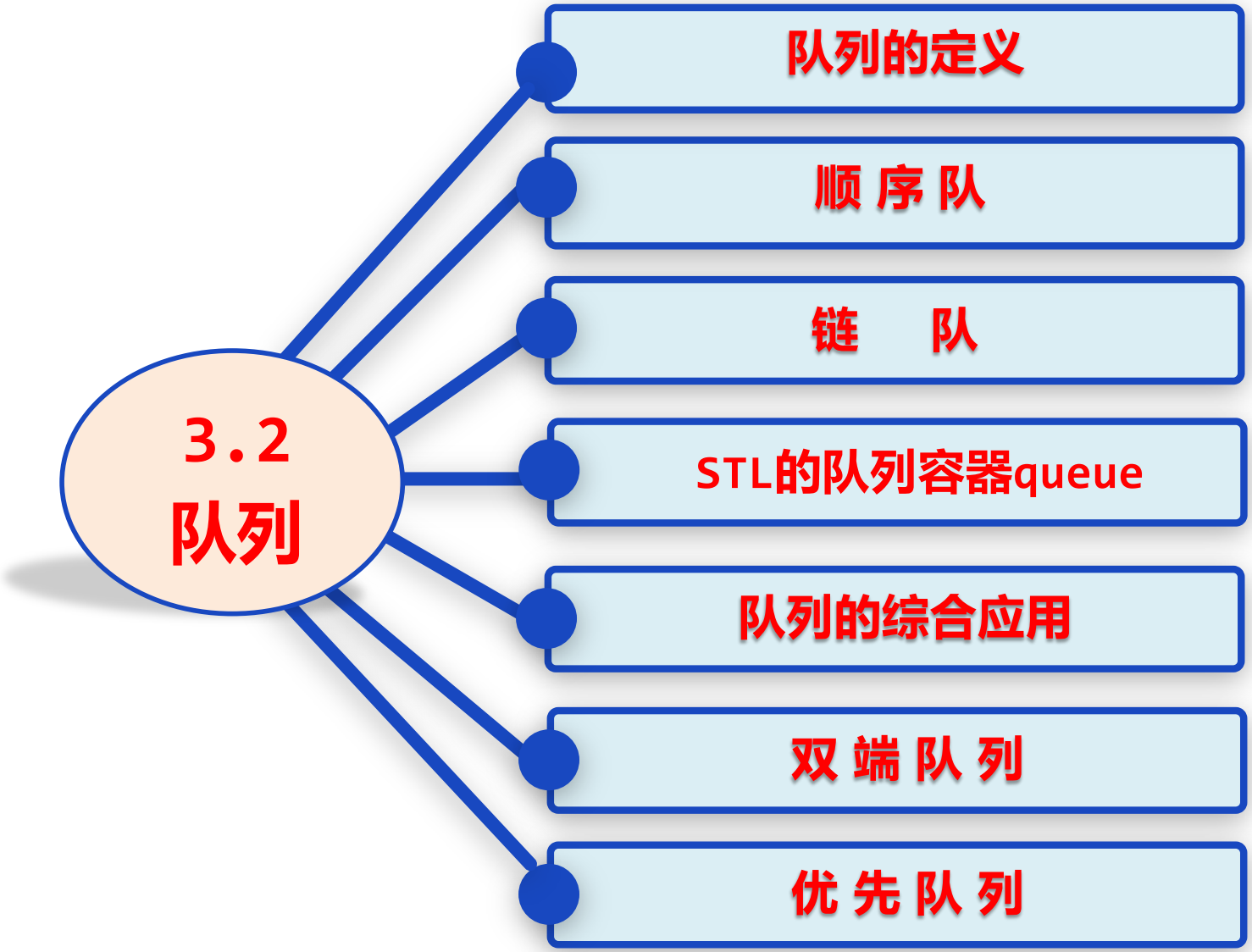
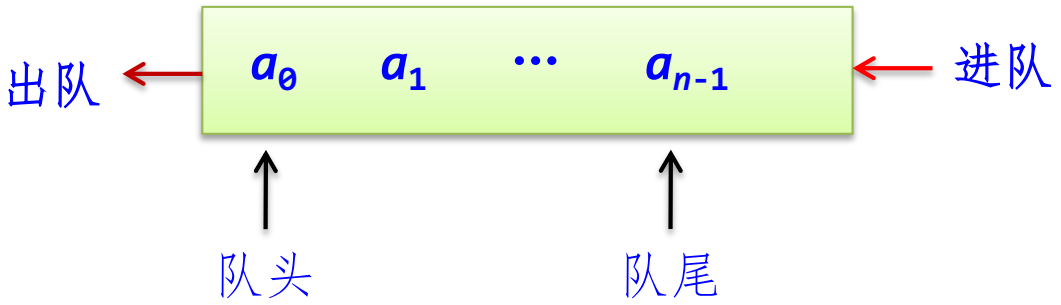
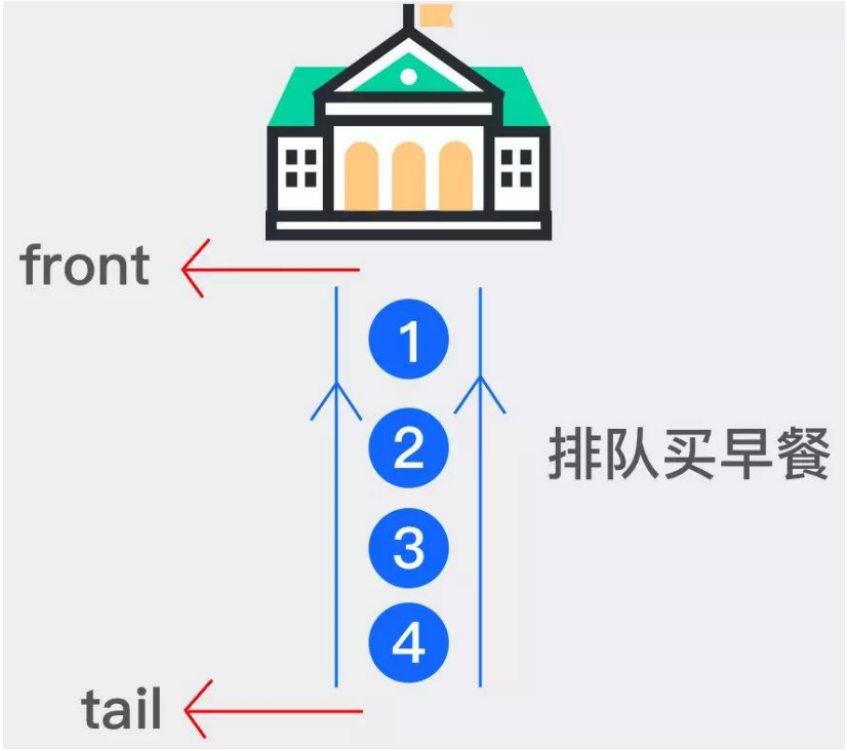

3.2
队列
队列的定义
顺 序 队
链 队
STL的队列容器queue
队列的综合应用
双 端 队 列
优 先 队 列

3.2.1 队列的定义

- 队列（queue）是一种只能在不同端进行插入或删除操作的线性表。
- 进行插入的一端称做队尾（rear），进行删除的一端称做队头或队首（front）。
- 队列的插入操作通常称为进队或入队（push），队列的删除操作通常称为出队或离队（pop）。





先买餐的人先出队

队列的主要特点：

- 先进先出，即先进队的元素先出队。
- 每次进队的元素作为新队尾元素，每次出队的元素只能是队头的元素。
- 队列也称为先进先出表。

ADT Queue

{

数据对象：

D={a_i | 0≤i≤n-1, n≥0}

数据关系：

R={r}

r={<a_i, a_{i+1}> | a_i, a_{i+1}∈D, i=0, …, n-2}

基本运算：

empty()：判断队列是否为空，若队列为空，返回真，否则返回假。

push(T e)：进队，将元素e进队作为队尾元素。

pop(T& e)：出队，从队头出队一个元素。

gethead(T& e)：取队头，返回队头元素而不出队。

}

队列抽象数据类型 = 线性结构 + 队列的基本运算

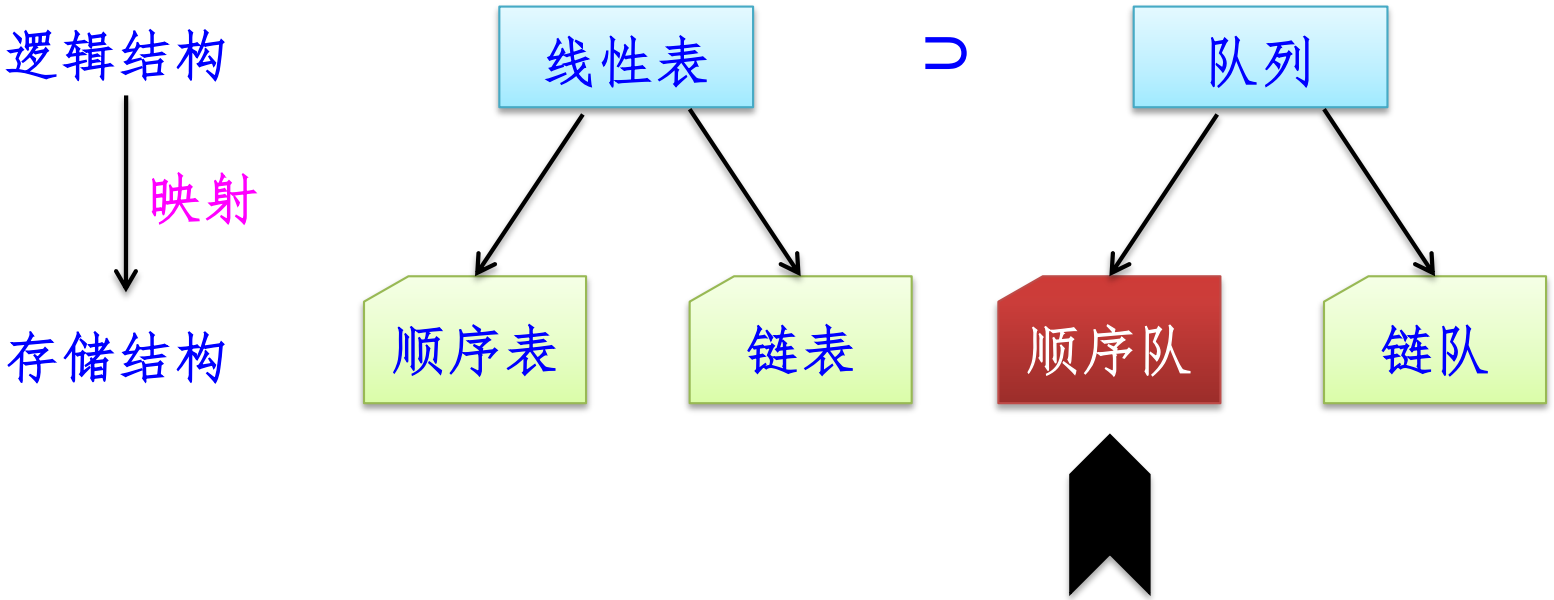
【例3.10】若元素进队顺序为1234，能否得到3142的出队序列？



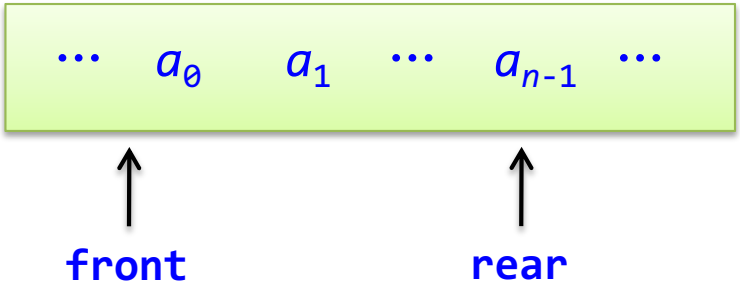
进队顺序为1234，则出队的顺序也为1234（先进先出），所以不能得到3142的出队序列。

3.2.2 队列的顺序存储结构及其基本运算算法实现

队列的实现方式



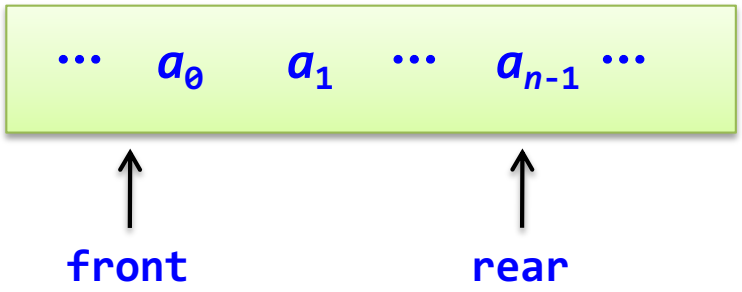
- 用data数组来存放队列中元素。
- 约定队头指针为front（实际上是队头元素的前一个位置），队尾指针为rear（正好是队尾元素的位置）。

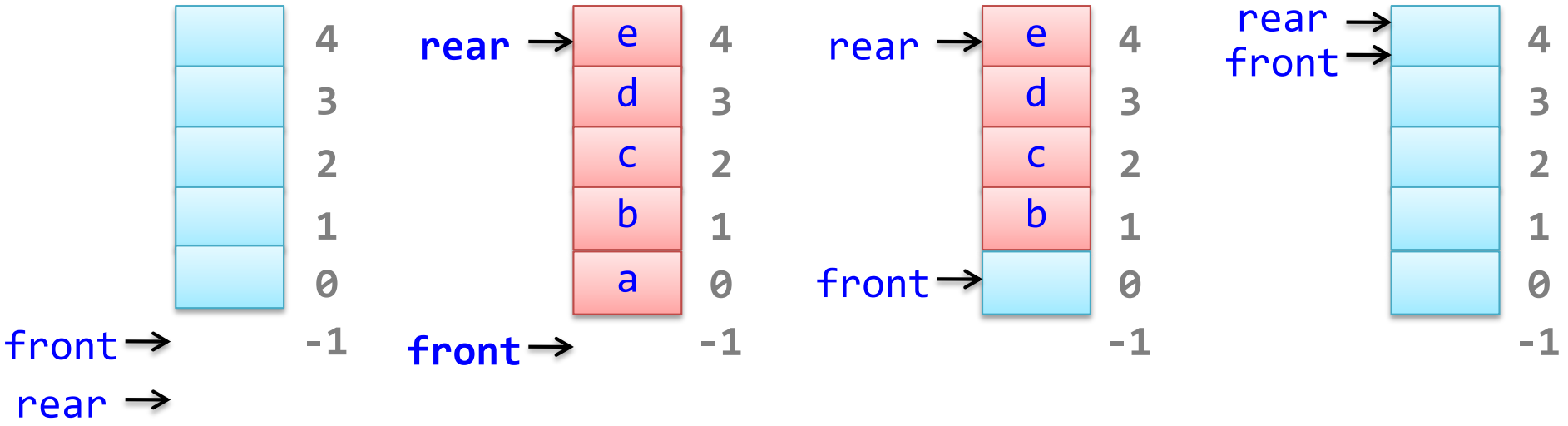


为了简单，使用固定容量的数组data（容量为常量MaxSize）。

1．非循环队列

- 初始时置front和rear均为-1（front==rear）
- 元素进队，rear增加1
- 元素出队列，front增加1





（a）空队　（b）5个元素进队　（c）出队1次　（c）出队4次

初始时置front和rear均为-1（front==rear），该顺序队的四要素如下：

- 队空条件：front==rear。
- 队满（上溢出）条件：rear==MaxSize-1（因为每个元素进队都让rear增1，当rear到达最大下标时不能再增加。
- 元素e进队操作：rear增1，将元素e放在该位置（进队的元素总是在尾部插入的）。
- 出队操作：front增1，取出该位置的元素（出队的元素总是在队头出来的）。

非循环队列类SqQueue

```
const int MaxSize=100;                    //队列的容量
template <typename T>
class SqQueue                             //非循环队队列类模板
{
public:
   T* data;                               //存放队中元素
   int front, rear;                       //队头和队尾指针
   //队列的基本运算算法
};
```

非循环队列的基本运算算法

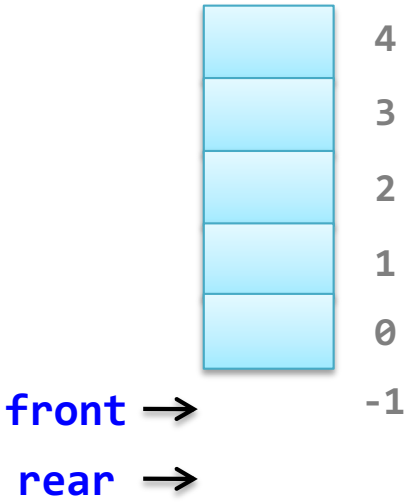
（1）非循环队列的初始化和销毁

```
SqQueue()                          //构造函数
{   data=new T[MaxSize];           //为data分配容量为MaxSize的空间
    front=rear=-1;                 //队头队尾指针置初值
}

~SqQueue()                         //析构函数
{
    delete [] data;
}
```

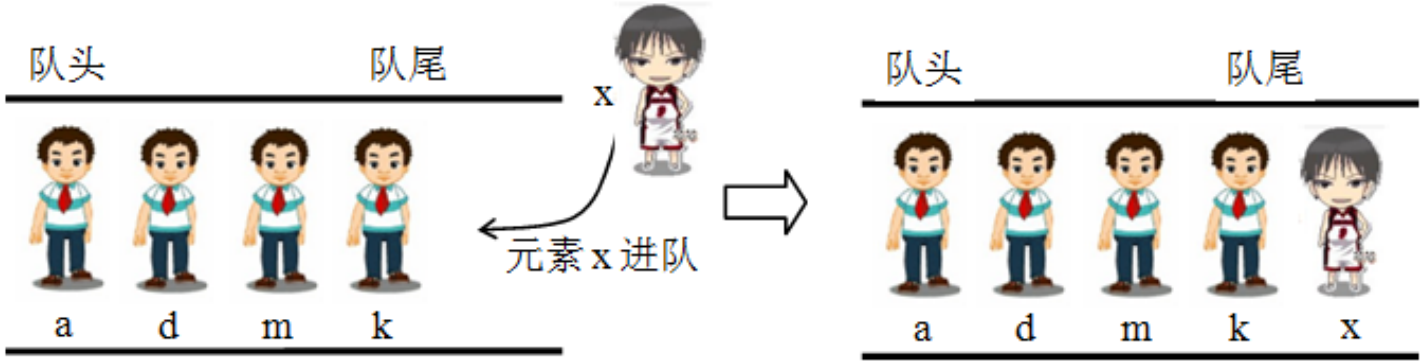
（2）判断队列是否为空empty()

```
bool empty()                    //判队空运算
{
    return (front==rear);
}
```

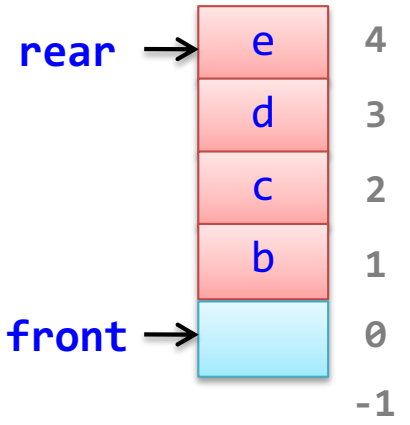


（3）进队push(T e)

元素e进队只能从队尾插入，不能从队头或中间位置进队,仅仅改变队尾指针



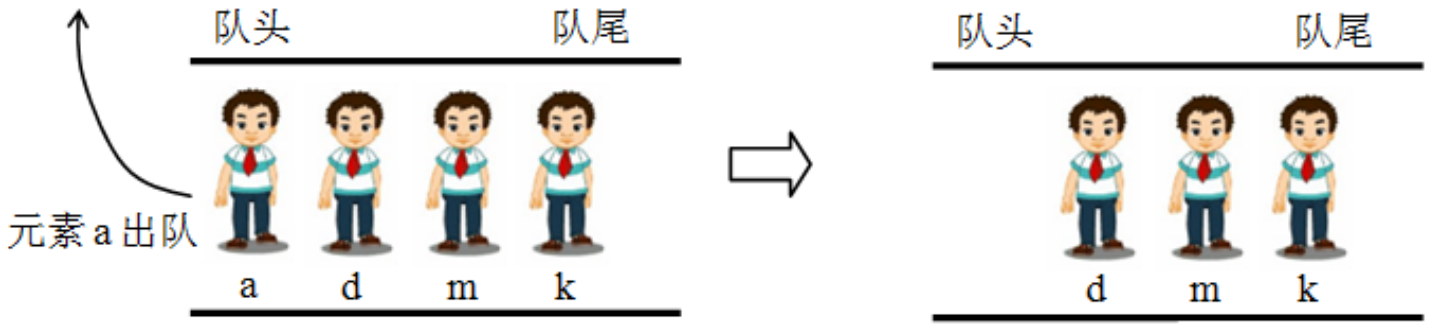
```
bool push(T e)                          //进队列运算
{   if (rear==MaxSize-1)                //队满上溢出
        return false;
    rear++;
    data[rear]=e;
    return true;
}
```



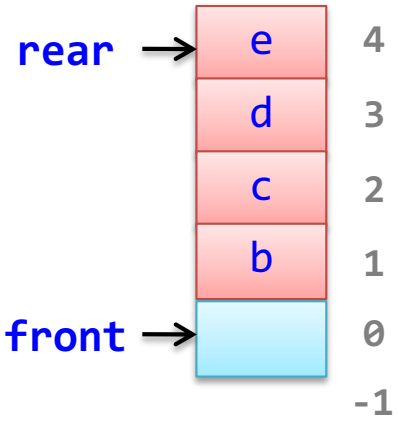
不能进队元素 ← **假溢出！**

（4）出队pop(T& e)

元素出队只能从队头删除，不能从队头或中间位置出队，仅仅改变队头指针

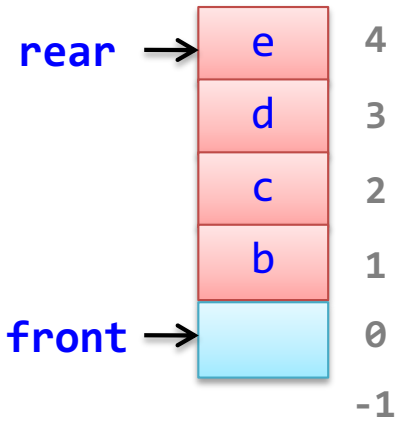


```
bool pop(T& e)                          //出队列运算
{  if (front==rear)                     //队空下溢出
       return false;
   front++;
   e=data[front];
   return true;
}
```



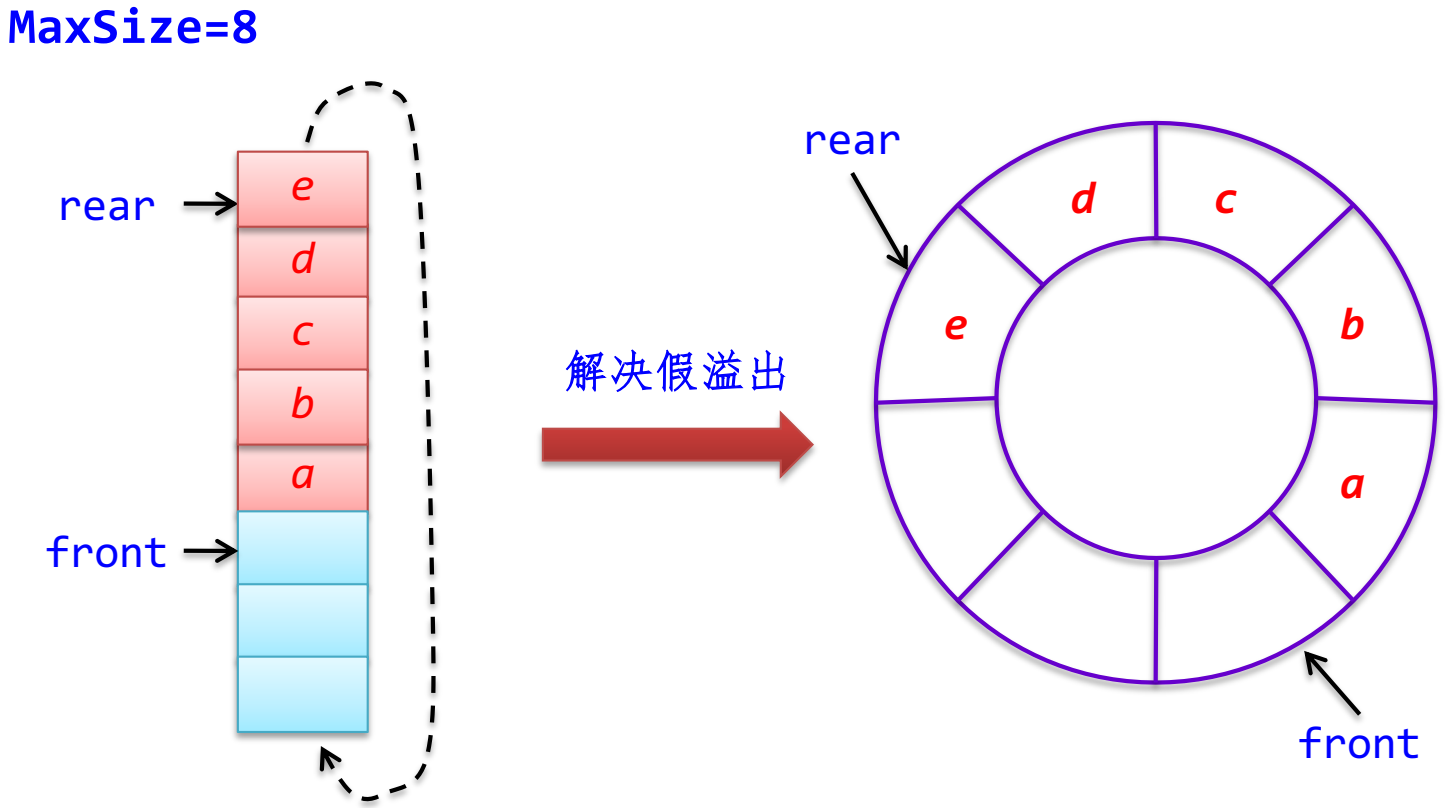
(5) 取队头元素gethead()

```
bool gethead(T& e)                    //取队头运算
{  if (front==rear)                   //队空下溢出
      return false;
   int head=front+1;
   e=data[head];
   return true;
}
```



2．循环队列

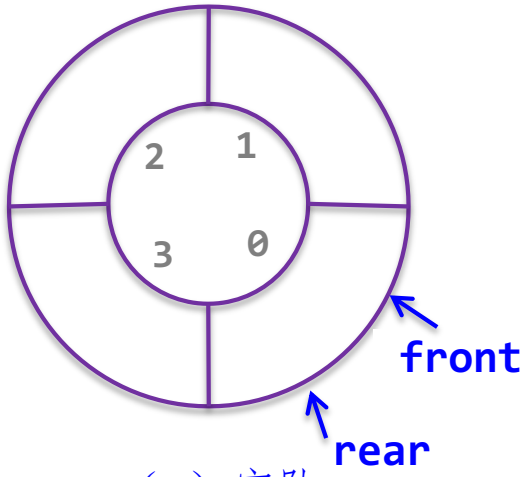
把data数组的前端和后端连接起来，形成一个循环数组，即把存储队列元素的表从逻辑上看成一个环，称为循环队列（也称为环形队列）。



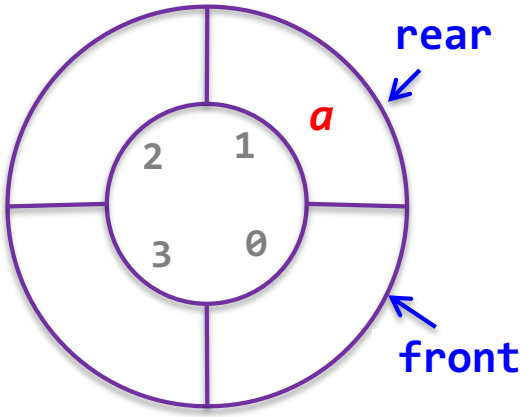
循环队列首尾相连，当队尾指针**rear=MaxSize-1**时，再前进一个位置就应该到达**0**位置，这可以利用数学上的求余运算（%）实现：

- 队首指针循环进**1**：**front=(front+1)%MaxSize**
- 队尾指针循环进**1**：**rear=(rear+1)%MaxSize**

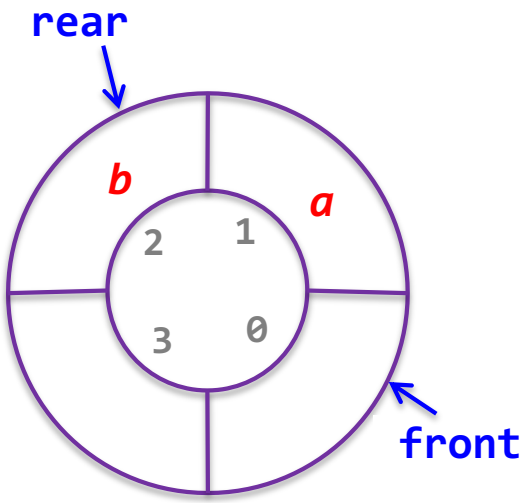
MaxSize=4，初始front=rear=0



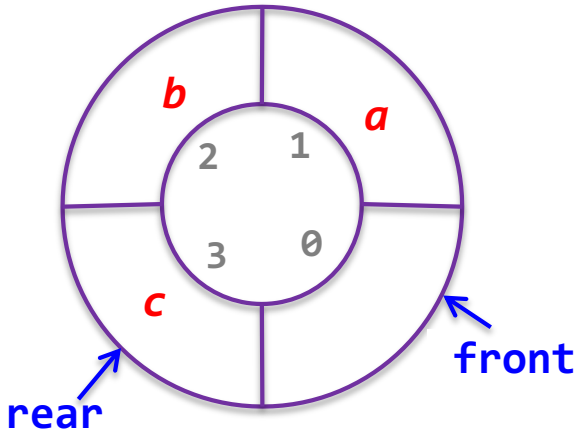
（a）空队



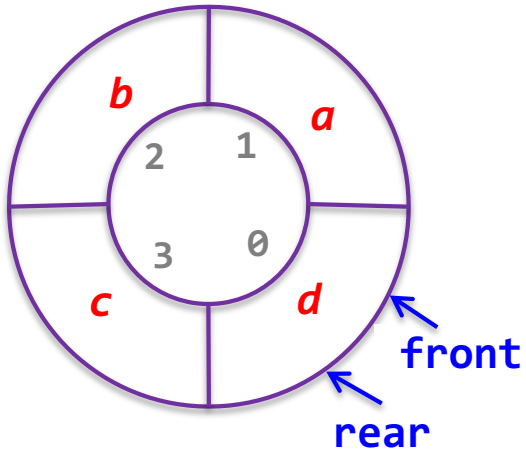
（b）a进队



（c）b进队



（d）c进队



（e）d进队

问题：如何区分队空和队满足?

如何设计队空队满的条件?

- 顺序队（含循环队列和非循环队列）通过front和rear标识队列状态，一般是采用它们的相对值即|front-rear|实现的。
- 若data数组的容量为*m*，则队列的状态有*m*+1种，分别是队空、队中有1个元素、队中有2个元素、…、队中有*m*个元素（队满）。
- front和rear的取值范围均为0～*m*-1，这样|front-rear|只有*m*个值。
- 显然*m*+1种状态不能直接用|front-rear|区分，因为必定有两种状态不能区分。
- 为此让队列中最多只有*m*-1个元素，这样队列恰好只有*m*种状态了，就可以通过front和rear的相对值区分所有状态了。

在规定队列中最多只有*m*-1个元素时，设置队空条件仍然是rear==front。当队列有*m*-1个元素时一定满足(rear+1)%MaxSize==front。

这样，循环队列在初始时置front=rear=0，其四要素如下：

- 队空条件：rear==front。
- 队满条件：(rear+1)%MaxSize==front（相当于试探进队一次，若rear达到front，则认为队满了）。
- 元素*e*进队：rear=(rear+1)%MaxSize，将元素*e*放置在该位置。
- 元素出队：front=(front+1)%MaxSize，取出该位置的元素。

循环队列类模板SqQueue

```
const int MaxSize=100;                          //队列的容量
template <typename T>
class CSqQueue                                  //循环队队列类模板
{
public:
    T* data;                                    //存放队中元素
    int front, rear;                            //队头和队尾指针
    //队列的基本运算算法
};
```

循环队列的基本运算算法

（1）循环队列的初始化和销毁

```
CSqQueue()                          //构造函数
{   data=new T[MaxSize];            //为data分配容量为MaxSize的空间
    front=rear=0;                   //队头队尾指针置初值
}

~CSqQueue()                         //析构函数
{
    delete [] data;
}
```

（2）判断队列是否为空empty()

```
bool empty()              //判队空运算
{
   return (front==rear);
}
```

（3）进队push(T e)

```
bool push(T e)                          //进队列运算
{  if ((rear+1)%MaxSize==front)         //队满上溢出
      return false;
   rear=(rear+1)%MaxSize;
   data[rear]=e;
   return true;
}
```

（4）出队pop(T& e)

```
bool pop(T& e)                    //出队列运算
{   if (front==rear)              //队空下溢出
       return false;
    front=(front+1)%MaxSize;
    e=data[front];
    return true;
}
```

（5）取队头元素gethead(T& e)

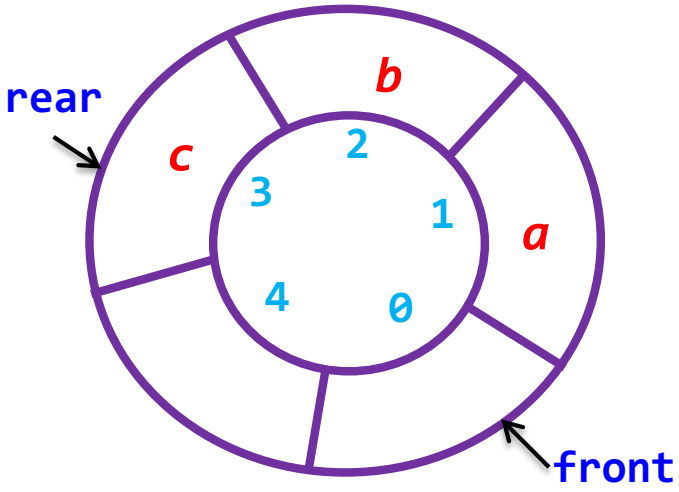
```
bool gethead(T& e)                    //取队头运算
{   if (front==rear)                  //队空下溢出
       return false;
    int head=(front+1)%MaxSize;
    e=data[head];
    return true;
}
```

3.2.3 顺序队的应用算法设计示例

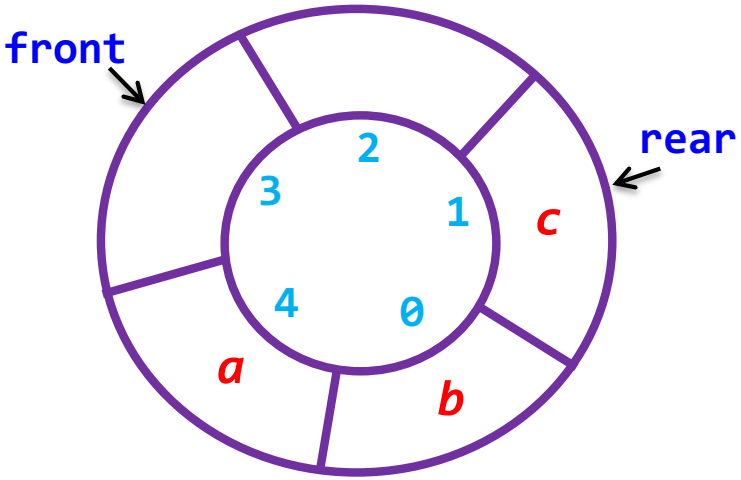
【例3.11】在CSqQueue循环队列类中增加一个求元素个数的算法getlength()。对于一个整数循环队列qu，利用队列基本运算和getlength()算法设计进队和出队第k（$k\geqslant 1$，队头元素的序号为1）个元素的算法。



MaxSize=5



cnt=(rear-front)=3



cnt=(rear-front)=-2 ×

cnt=(rear-front+MaxSize)=3

cnt=(rear-front+MaxSize)=8 ×

cnt=(rear-front+MaxSize)%MaxSize=3 √

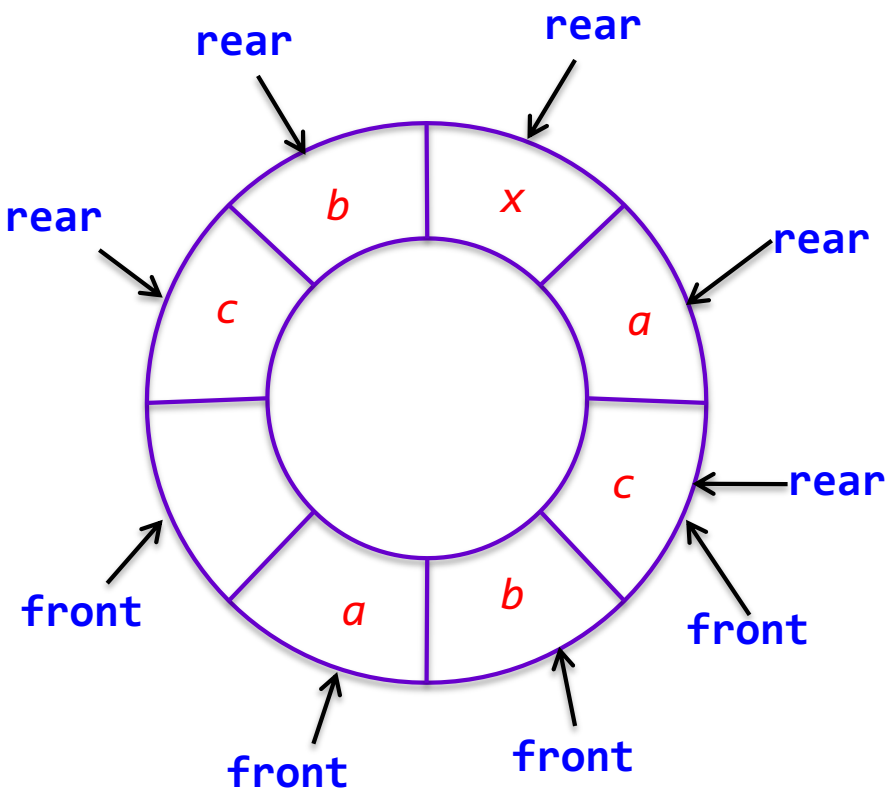
cnt=(rear-front+MaxSize)%MaxSize=3 √

在CSqQueue循环队列类中增加getlength()算法如下：

```
int getlength()                    //返回队中元素个数
{
   return (rear-front+MaxSize)%MaxSize;
}
```

进队第*k*（*k*≥1）个元素e

例如： *e*='*x*', *k*=2：*a b* c ⇨ *a x b c*



操作完毕

进队第*k*（*k*≥1）个元素*e*的算法：

```
bool pushk(CSqQueue<int>& qu,int k,int e) //进队第k个元素e
{  int x;
   int n=qu.getlength();
   if (k<1 || k>n+1)
      return false;                      //参数k错误返回False
   if (k<=n)
   {  for (int i=1;i<=n;i++)             //循环处理队中所有元素
      {  if (i==k)
            qu.push(e);                  //将e元素进队到第k个位置
         qu.pop(x);                      //出队元素x
         qu.push(x);                     //进队元素x
      }
   }
   elsequ.push(e);                       //k=n+1时直接进队e
   return true;
}
```

出队第*k*（*k*≥1）个元素e的算法思路：出队前*k*-1个元素，边出边进，出队第*k*个元素e，*e*不进队，将剩下的元素边出边进。

```
bool popk(CSqQueue<int>& qu,int k,int& e)   //出队第k个元素
{  int x;
   int n=qu.getlength();
   if (k<=1 || k>n) return false;     //参数k错误返回false
   for (int i=1;i<=n;i++)             //循环处理队中所有元素
   {  qu.pop(x);                      //出队元素x
      if (i!=k)
         qu.push(x);                  //将非第k个元素进队
      else
         e=x;                         //取第k个出队的元素
   }
   return true;
}
```

【例3.12】对于循环队列来说，如果知道队头指针和队列中元素个数，则可以计算出队尾指针。也就是说，可以用队列中元素个数代替队尾指针。设计出这种循环队列的判队空、进队、出队和取队头元素的算法。

count=(rear-front+MaxSize)%MaxSize



已知front、count，求rear：

rear=(front+count)%MaxSize

已知rear、count，求front：

front=(rear-count+MaxSize)%MaxSize

对应的循环队列类模板CSqQueue1

```
const int MaxSize=100;                    //队列的容量
template <typename T>
class CSqQueue1                           //循环队队列类模板
{
public:
   T* data;                               //存放队中元素
   int front;                             //队头指针
   int count;                             //队中元素个数

   CSqQueue1()                            //构造函数
   {  data=new T[MaxSize];                //为data分配容量为MaxSize的空间
      front=0;                            //队头指针置初值
      count=0;                            //元素个数置初值
   }

   ~CSqQueue1()                           //析构函数
   {
      delete [] data;
   }
```

```
//----------循环队基本运算算法-----------------
bool empty()                                    //判队空运算
{
   return count==0;
}

bool push(T e)                                  //进队列运算
{  if (count==MaxSize)                          //队满上溢出
     return false;
   int rear1=(front+count)%MaxSize;             //求队尾（rear1为局部变量）
   rear1=(rear1+1) % MaxSize;                   //队尾循环进1
   data[rear1]=e;
   count++;                                     //元素个数增1
   return true;
}
```

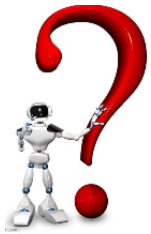
方法中的局部变量

```
    bool pop(T& e)                          //出队列运算
    {   if (count==0)                       //队空下溢出
            return false;
        front=(front+1)%MaxSize;
        e=data[front];
        count--;                            //元素个数减少1
        return true;
    }

    bool gethead(T& e)                      //取队头运算
    {   if (count==0)                       //队空下溢出
            return false;
        int head=(front+1)%MaxSize;
        e=data[head];
        return true;
    }
};
```



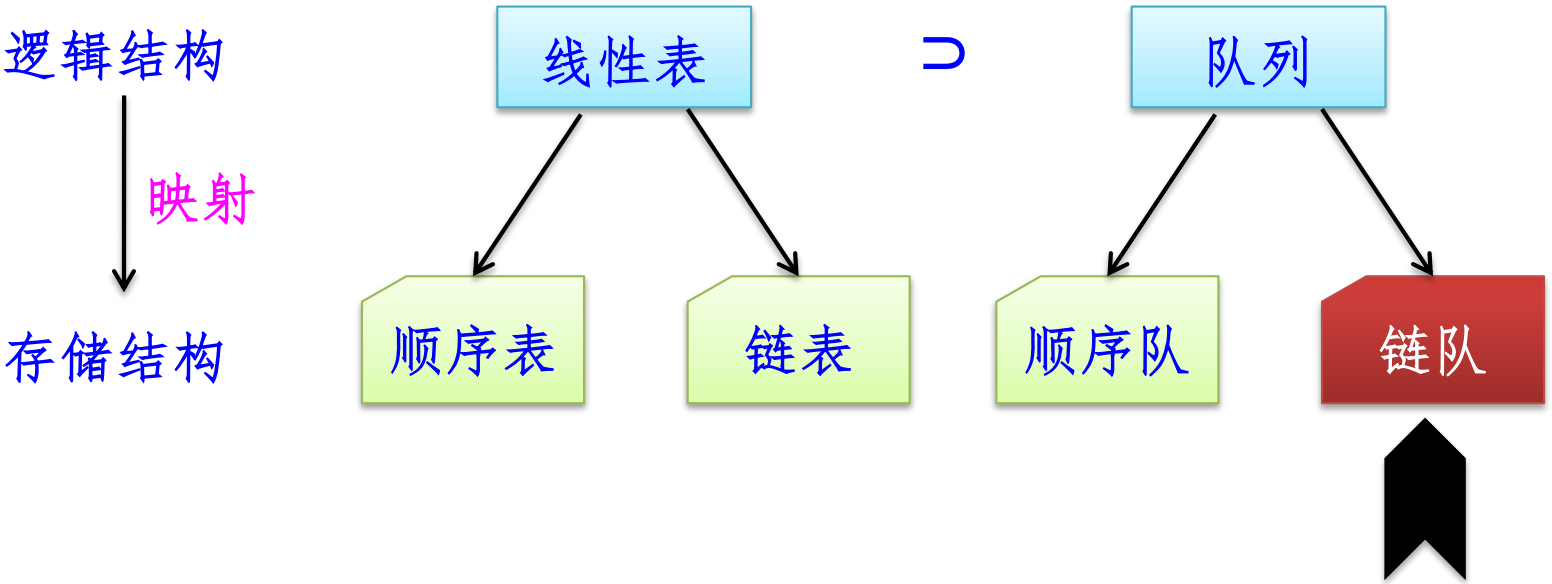
本例设计的循环队列中最多可保存MaxSize个元素。

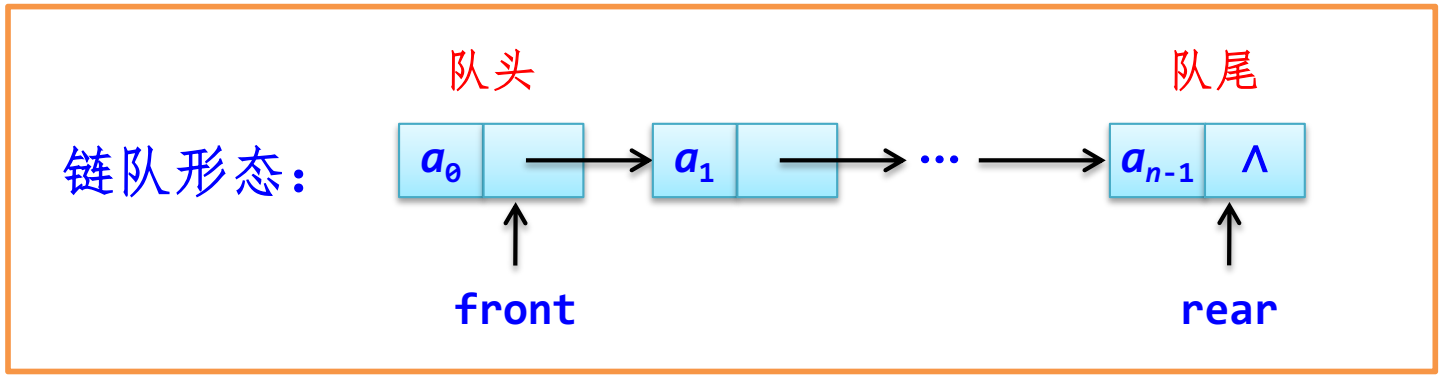


如果将顺序队改为容量可以扩展的，如何设计？

3.2.4 队列的链式存储结构及其基本运算算法实现

队列的实现方式

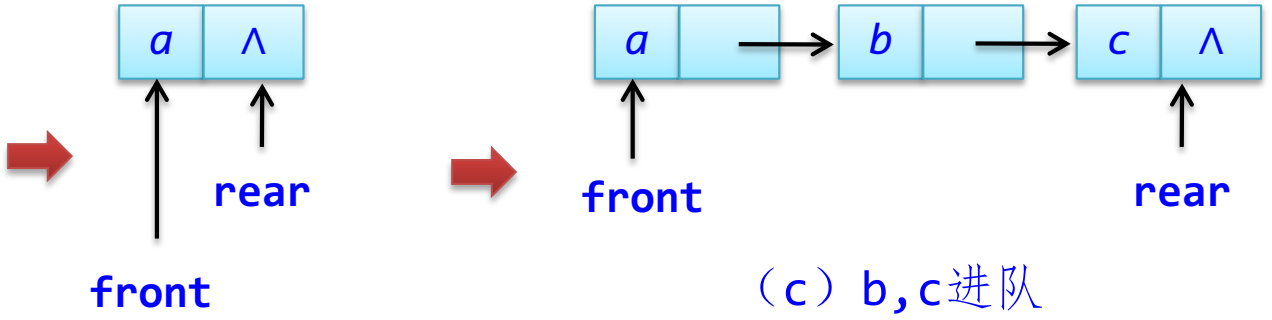




操作示例

front=NULL

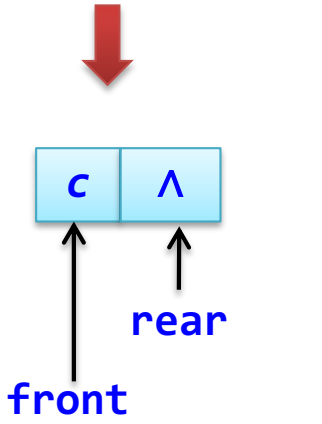
rear=NULL



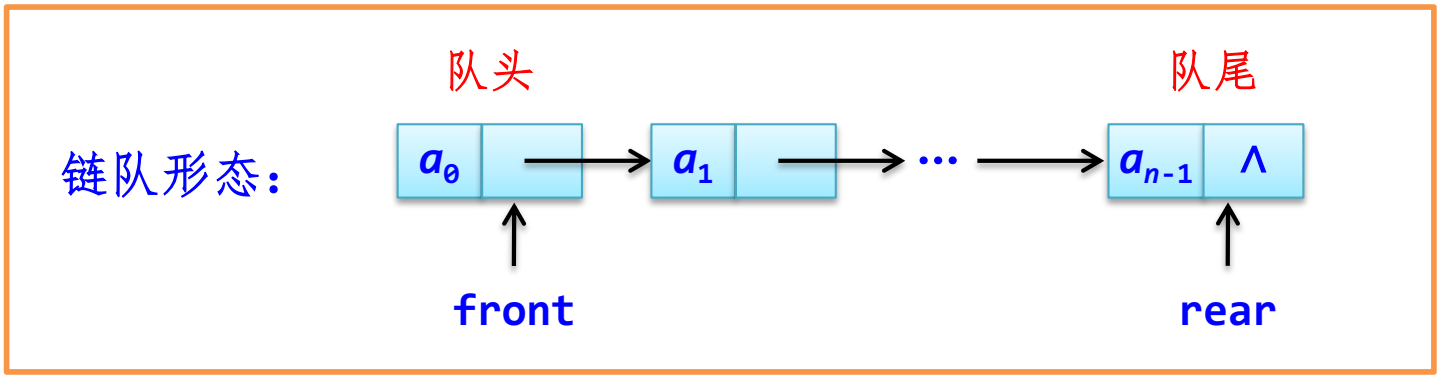
（a）链队初态

（b）a进队

（c）b,c进队



（d）出队两次



初始时置front=rear=NULL。链队的四要素如下：

- 队空条件:front=rear=NULL，不妨仅以rear==NULL作为队空条件。
- 由于只有内存溢出时才出现队满，通常不考虑这样的情况。
- 元素e进队操作：在单链表尾部插入存放e的s结点，并让队尾指针指向它。
- 出队操作：取出队首结点的data值并将其从链队中删除。

和单链表一样，链队中每个结点的类型LinkNode如下

```
template <typename T>
struct LinkNode                                  //链队数据结点类型
{   T data;                                      //结点数据域
    LinkNode* next;                              //指向下一个结点

    LinkNode():next(NULL) {}                     //构造函数
    LinkNode(T d):data(d),next(NULL) {}          //重载构造函数
};
```

3.2.5 链队的应用算法设计示例

【例3.13】采用链队求解第2章例2.16的约瑟夫问题。



【例2.16】编写一个程序求解约瑟夫（Joseph）问题。有n个小孩围成一圈，给他们从1开始依次编号，从编号为1的小孩开始报数，数到第m个小孩出列，然后从出列的下一个小孩重新开始报数，数到第m个小孩又出列，…，如此反复直到所有的小孩全部出列为止，求整个出列序列。

如当n=6，m=5时的出列序列是5，4，6，2，3，1。



先定义一个链队qu：

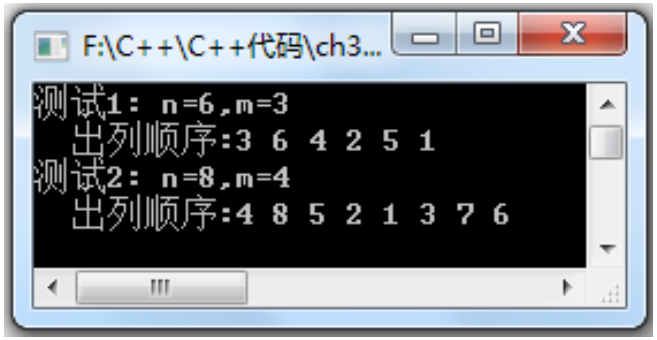
- 对于(*n*，*m*)约瑟夫问题，依次将1～*n*进队。
- 循环*n*次出列*n*个小孩：依次出队*m*-1次，将所有出队的元素立即进队（将他们从队头出队后插入到队尾），再出队第*m*个元素并且输出（出列第*m*个小孩）。

```
#include "LinkQueue.cpp"                    //包含链队类模板的定义
void Jsequence(int n,int m)                 //输出约瑟夫序列
{  int x;
   LinkQueue<int> qu;                       //定义一个链队
   for (int i=1;i<=n;i++)                   //进队编号为1到n的n个小孩
       qu.push(i);

   for (int i=1;i<=n;i++)                   //共出列n个小孩
   {  int j=1;
      while (j<=m-1)                        //出队m-1个小孩，并将他们进队
      {  qu.pop(x);
         qu.push(x);
         j++;
      }
      qu.pop(x);                            //出队第m个小孩
      cout << x << " ";
   }
   cout << endl;
}
```

程序验证

```
int main()
{  printf("测试1: n=6,m=3\n");
   printf("  出列顺序:");
   Jsequence(6,3);
   printf("测试2: n=8,m=4\n");
   printf("  出列顺序:");
   Jsequence(8,4);
   return 0;
}
```



3.2.6 STL中的queue队列容器

- STL中的queue队列容器具有先进先出的特点。queue容器只有一个进口即队尾（back）和一个出口即队头（front），可以在队尾插入（进队）元素，在队头删除（出栈）元素，不允许像数组那样从前向后或者从后向前顺序遍历。
- 与stack栈容器一样，queue队列容器也是一种适配器容器，其底层容器必须提供front()、back()、push_back()和pop_front()等操作，因此queue的底层容器可以是deque（默认）或者list，不能是vector，因为vector容器没有提供头部操作函数（如pop_front()）。

定义3个queue对象

```
queue<int> qu1;                 //定义一个整数队列qu1
queue<int> qu2(qu1);            //由qu1队列复制产生qu2队列
queue<int, list<int>> qu3;      //定义整数队列st3，以list作为底层容器
```

queue的主要成员函数及其说明

成员函数	说明
empty()	判断队列容器是否为空
size()	返回队列容器中实际元素个数
front()	返回队头元素
back()	返回队尾元素
push(e)	元素e进队
pop()	元素出队

```
#include<iostream>
#include <queue>
using namespace std;
int main()
{  queue<int> qu;
   qu.push(1); qu.push(2); qu.push(3);
   printf("队头元素: %d\n",qu.front());
   printf("队尾元素: %d\n",qu.back());
   printf("出队顺序: ");
   while (!qu.empty())                          //出队所有元素
   {  printf("%d ",qu.front());
      qu.pop();
   }
   printf("\n");
   return 0;
}
```



```
队头元素: 1
队尾元素: 3
出队顺序: 1 2 3
```

3.2.7 队列的综合应用

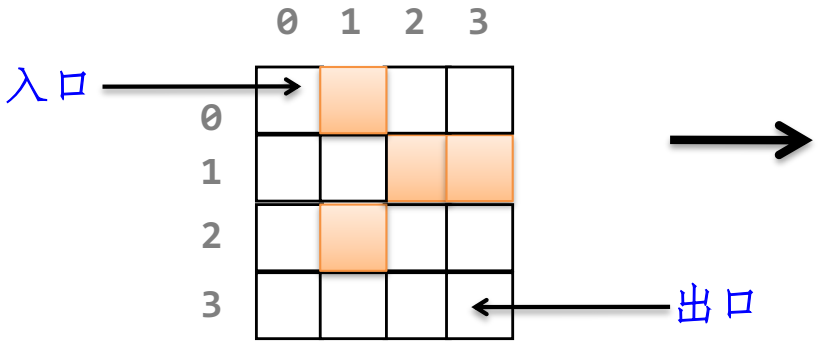
求解问题中需要临时保存一些数据元素：

- 先保存的后处理：栈
- 先保存的先处理：队列

满足先来先服务特性的应用均可采用队列作为其数据组织方式或中间数据结构

- 调度或缓冲
  - ✓ 消息缓冲器
  - ✓ 邮件缓冲器
  - ✓ 数据缓冲
  - ✓ 计算机的硬件设备间通信也需要队列作为数据缓冲
  - ✓ 操作系统的资源管理
- 宽度优先搜索

求迷宫问题

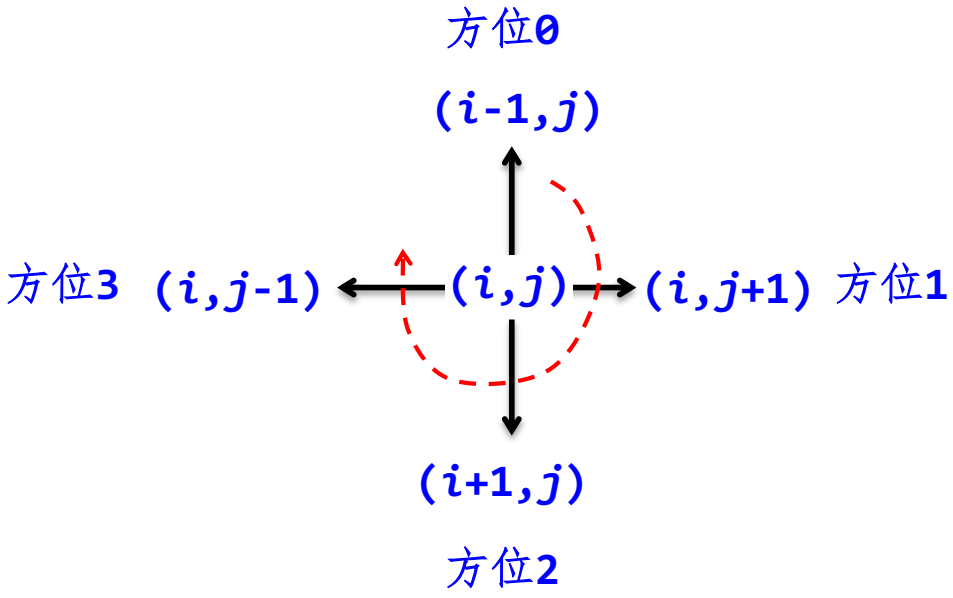



```
int mg[MAX][MAX]=
{{0,1,0,0},
 {0,0,1,1},
 {0,1,0,0},
 {0,0,0,0}
};
int m=4,n=4;
```

一个迷宫图

求从入口到出口的一条简单路径

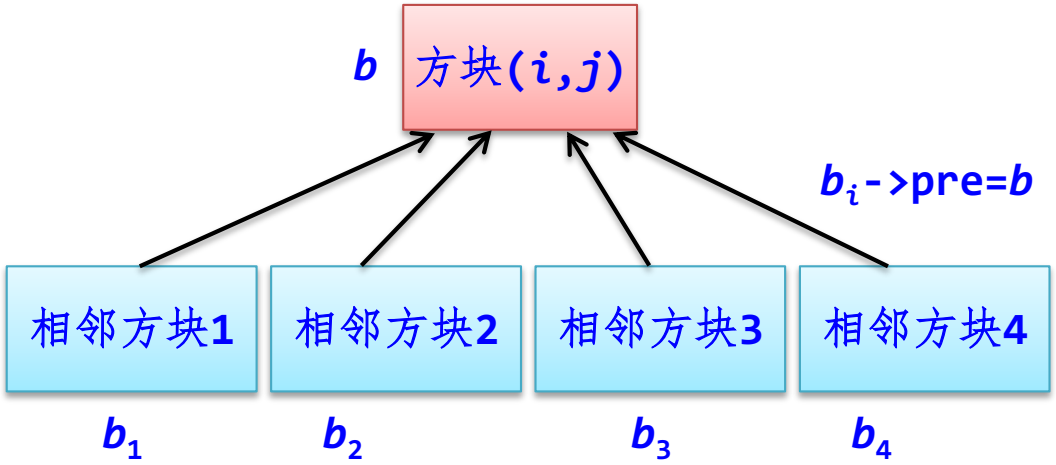
试探顺序




```
int dx[]={-1,0,1,0};          //x方向的偏移量
int dy[]={0,1,0,-1};          //y方向的偏移量
```

迷宫问题的搜索过程

- 每次走到一个方块（*i*,*j*），一次性试探所有相邻方块，将所有相邻可走方块进队。
- 一个方块在队列中的元素为*b*，并保存其前驱方块（pre标识）。
- 从入口开始，找到出口后由pre推导出迷宫路径。






```
struct Box                          //队列中方块元素类型
{   int i,j;                        //方块的行、列号
    Box* pre;                       //本路径中上一方块的地址

    Box() {}                        //构造函数
    Box(int i1,int j1)              //重载构造函数
    {   i=i1; j=j1;
        pre=NULL;
    }
};
```

```
bool mgpath(int xi,int yi,int xe,int ye)
//求一条从(xi,yi)到(xe,ye)的迷宫路径
{  Box* b,*b1;
   queue<Box* > qu;                    //定义一个队列qu
   b=new Box(xi,yi);                   //建立入口的对象b
   qu.push(b);                         //入口对象b进队,其pre置为NULL
   mg[xi][yi]=-1;                      //为避免来回找相邻方块置mg值为-1
   while (!qu.empty())                 //队不空时循环
   {  b=qu.front();                    //取队头方块b
      if (b->i==xe && b->j==ye)        //找到了出口,输出路径
      {  disppath(qu);                 //输出一条迷宫路径
         return true;                  //找到一条迷宫路径后返回true
      }
```

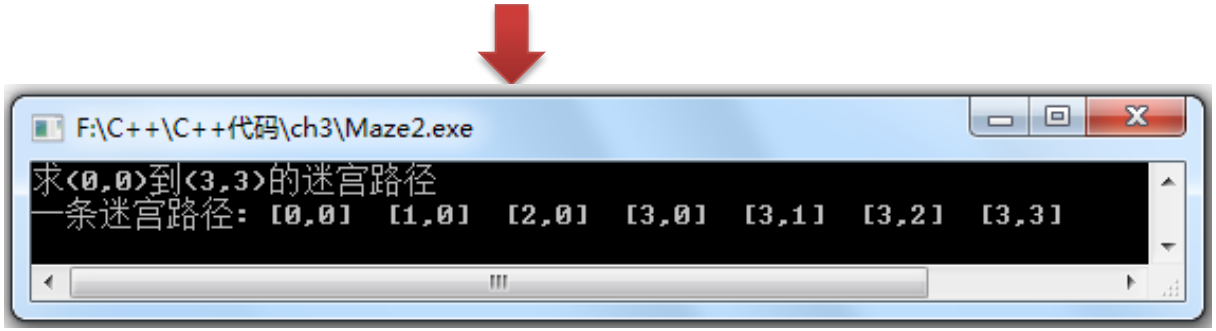
```
        qu.pop();                       //出队方块b
        for (int di=0;di<4;di++) //循环扫描每个方位,把每个可走的方块进队
        {  int i=b->i+dx[di];     //找b的di方位的相邻方块(i,j)
           int j=b->j+dy[di];
           if (i>=0 && i<m && j>=0 && j<n && mg[i][j]==0)
           {  b1=new Box(i,j);    //(i,j)方块有效且可走，建立其队列对象b1
              b1->pre=b;          //将该相邻方块进队,并置pre指向前驱方块
              qu.push(b1);
              mg[i][j]=-1;        //为避免来回找相邻方块置mg值为-1
           }
        }
    }
    return false;                   //未找到任何路径时返回false
}
```

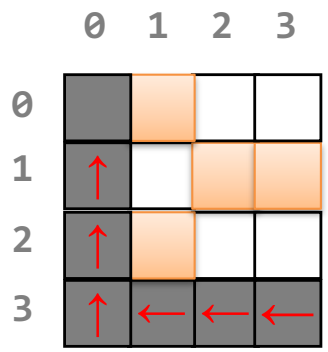
```
void disppath(queue<Box* >& qu)        //输出一条迷宫路径
{  vector<Box> apath;                  //存放一条迷宫路径
   Box* b;
   b=qu.front();                       //从队头开始向入口方向搜索
   while (b!=NULL)
   {  apath.push_back(Box(b->i,b->j));//将搜索的方块添加到apath中
      b=b->pre;
   }
   cout << "一条迷宫路径: ";
   for (int i=apath.size()-1;i>=0;i--)//反向输出构成一条正向迷宫路径
      cout << "[" << apath[i].i << "," << apath[i].j << "]  ";
   cout << endl;
}
```

设计主程序

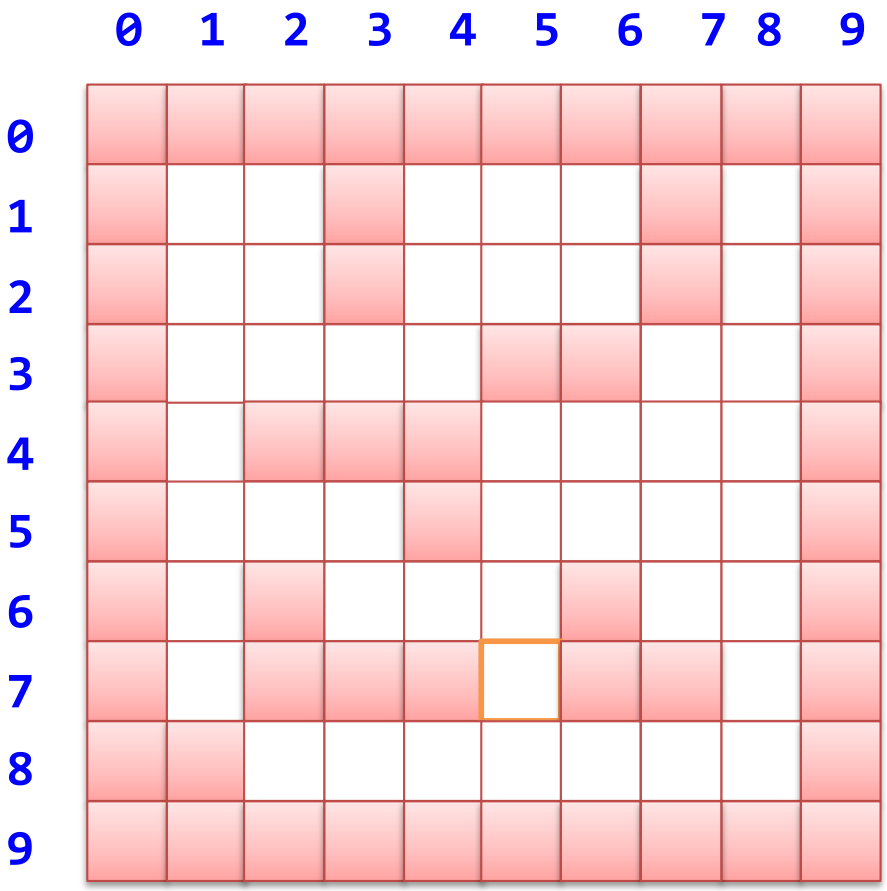
程序验证

```
int main()
{   int xi=0,yi=0,xe=3,ye=3;
    printf("求(%d,%d)到(%d,%d)的迷宫路径\n",xi,yi,xe,ye);
    if (!mgpath(xi,yi,xe,ye))
        cout << "不存在迷宫路径\n";
    return 0;
}
```





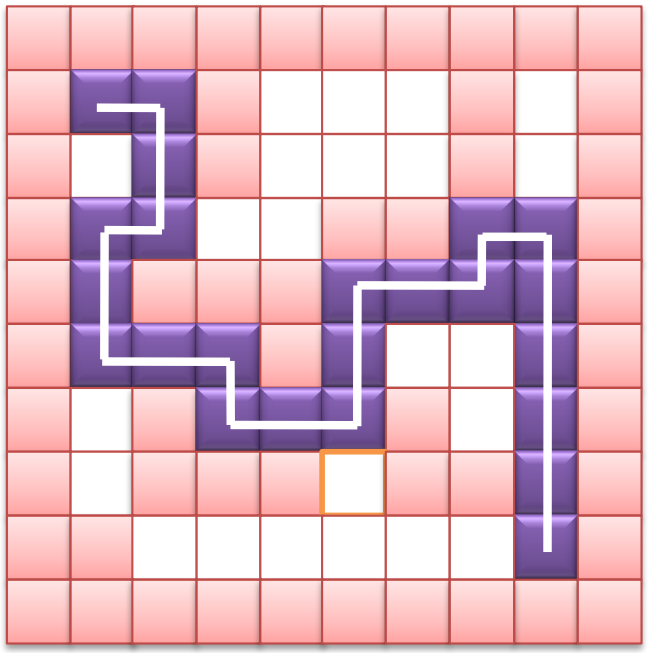
针对如图所示迷宫，演示用栈和队列找到的结果及过程



栈求解

迷宫路径如下：

(1, 1) (1, 2) (2, 2) (3, 2) (3, 1)
(4, 1) (5, 1) (5, 2) (5, 3) (6, 3)
(6, 4) (6, 5) (5, 5) (4, 5) (4, 6)
(4, 7) (3, 7) (3, 8) (4, 8) (5, 8)
(6, 8) (7, 8) (8, 8)

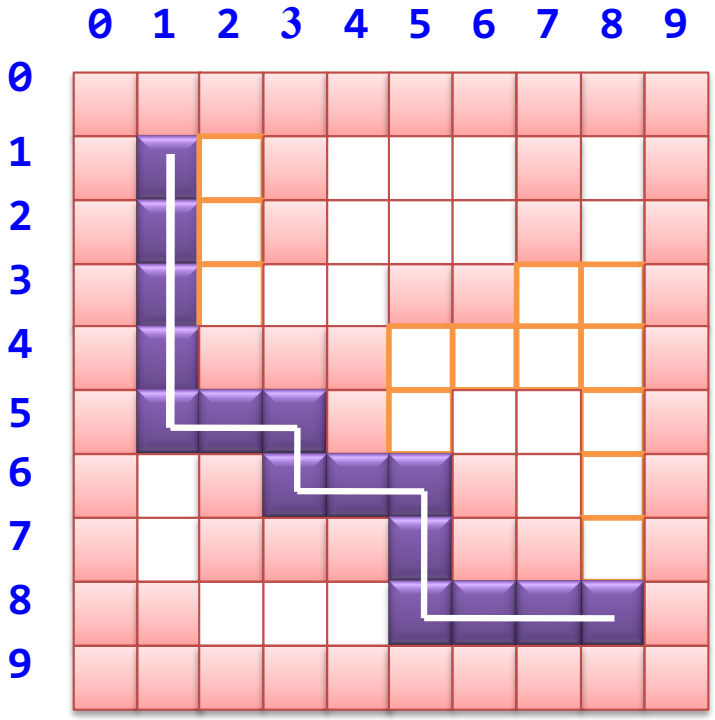


显然，这个解不是最优解，即不是最短路径。为什么？

队列求解

迷宫路径如下：

(1, 1) (2, 1) (3, 1) (4, 1) (5, 1)
(5, 2) (5, 3) (6, 3) (6, 4) (6, 5)
(7, 5) (8, 5) (8, 6) (8, 7) (8, 8)



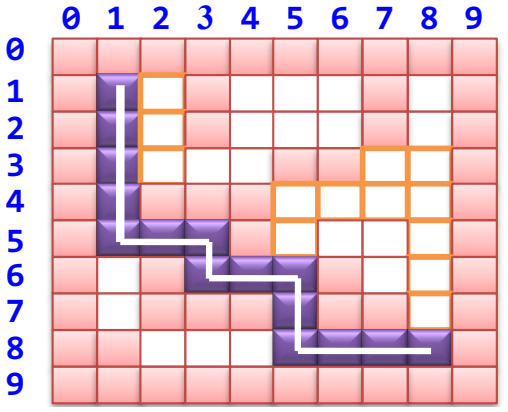
显然，这个解是最优解，即是最短路径。为什么？

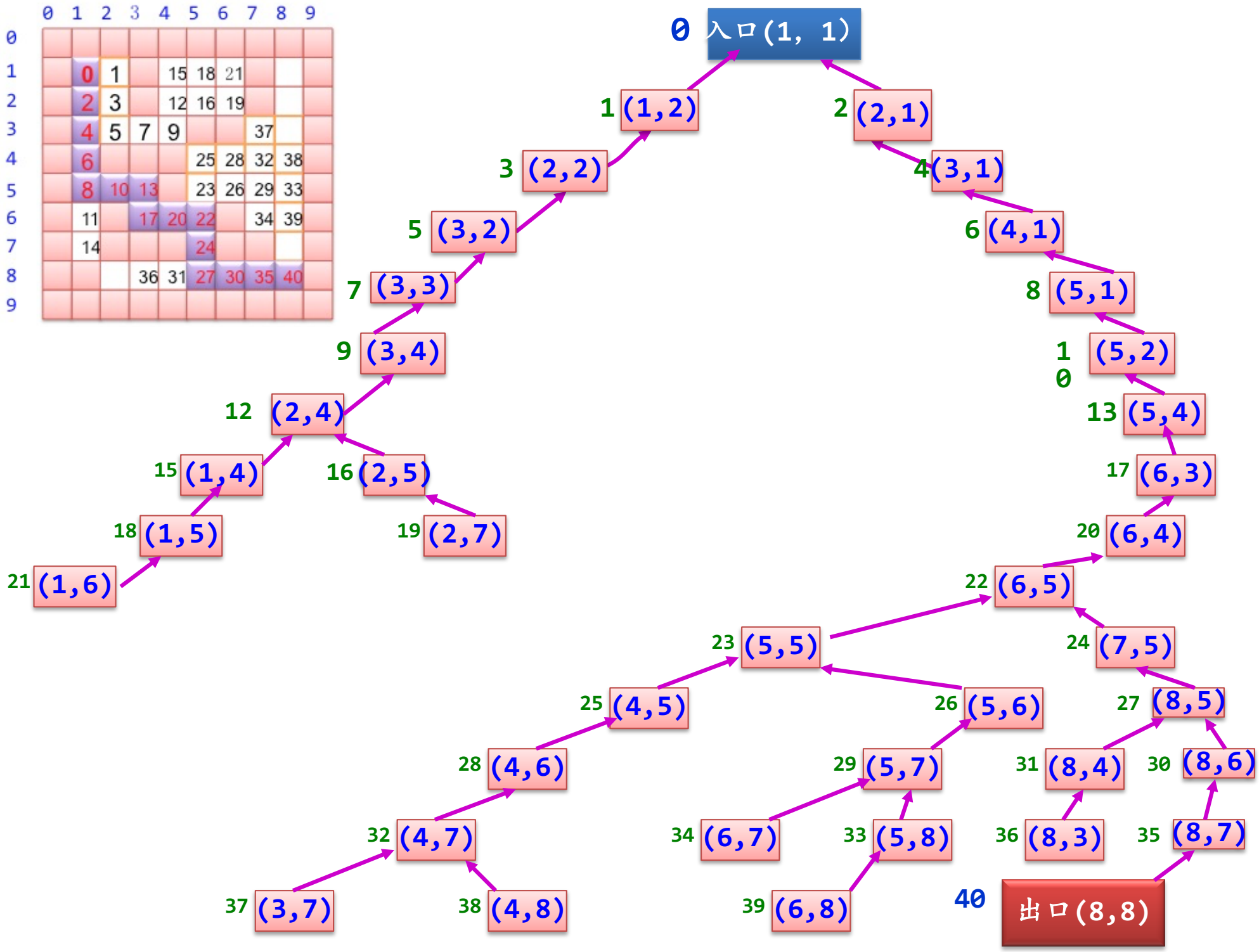
对于图3.11的迷宫，求解(1,1)→(8,8)时队列qu中结果如下：

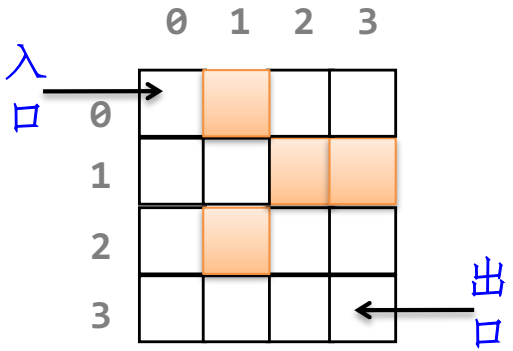
下标	i	j	pre
0	1	1	-1
1	1	2	0
2	2	1	0
3	2	2	1
4	3	1	2
5	3	2	3
6	4	1	4
7	3	3	5
8	5	1	6
9	3	4	7
10	5	2	8
11	6	1	8
12	2	4	9
13	5	3	10
14	7	1	11

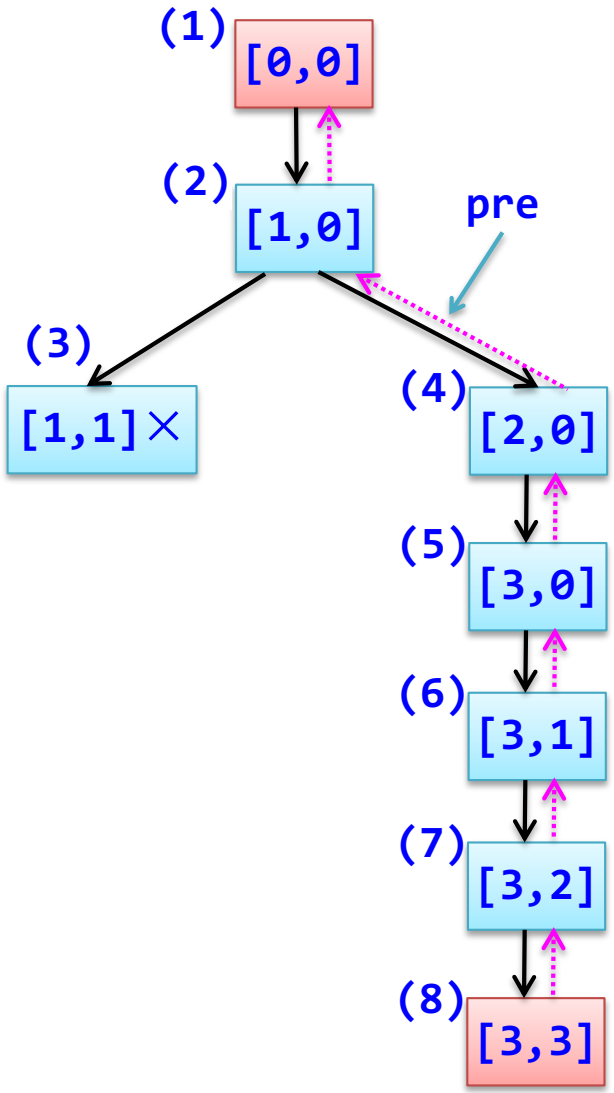
下标	i	j	pre
15	1	4	12
16	2	5	12
17	6	3	13
18	1	5	15
19	2	6	16
20	6	4	17
21	1	6	18
22	6	5	20
23	5	5	22
24	7	5	22
25	4	5	23
26	5	6	23
27	8	5	24
28	4	6	25
29	5	7	26

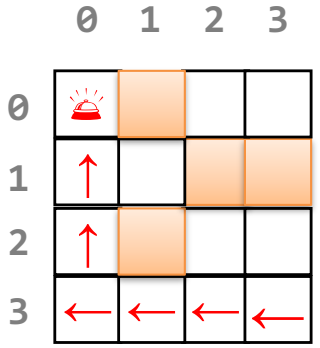
下标	i	j	pre
30	8	6	27
31	8	4	27
32	4	7	28
33	5	8	29
34	6	7	29
35	8	7	30
36	8	3	31
37	3	7	32
38	4	8	32
39	6	8	33
40	8	8	35

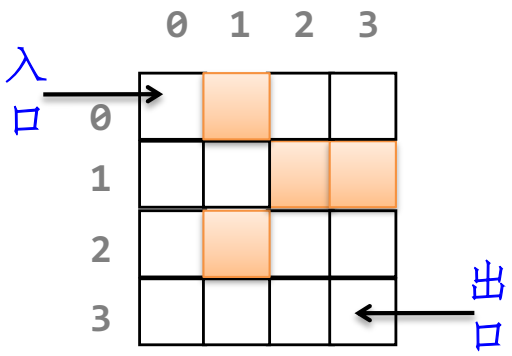


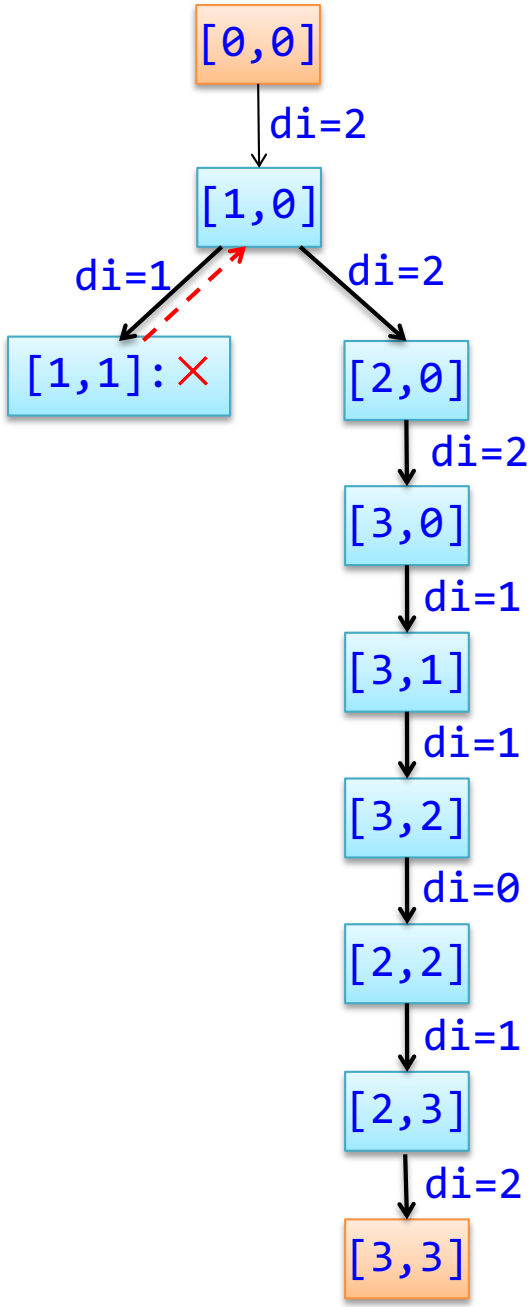

0 1 2 3 4 5 6 7 8 9
0 1 2 3 4 5 6 7 8 9
0 入口(1, 1)
1 (1,2)
2 (2,1)
3 (2,2)
4 (3,1)
5 (3,2)
6 (4,1)
7 (3,3)
8 (5,1)
9 (3,4)
10 (5,2)
12 (2,4)
13 (5,4)
15 (1,4)
16 (2,5)
17 (6,3)
18 (1,5)
19 (2,7)
20 (6,4)
21 (1,6)
22 (6,5)
23 (5,5)
24 (7,5)
25 (4,5)
26 (5,6)
27 (8,5)
28 (4,6)
29 (5,7)
31 (8,4)
30 (8,6)
32 (4,7)
34 (6,7)
33 (5,8)
36 (8,3)
35 (8,7)
37 (3,7)
38 (4,8)
39 (6,8)
40 出口(8,8)

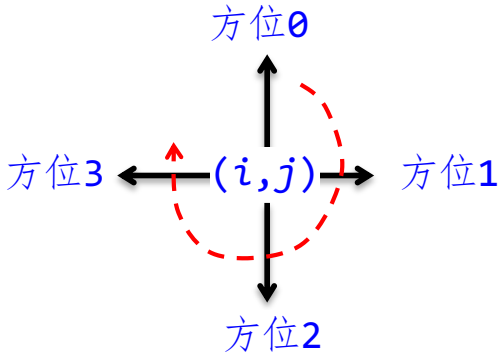

0 1 2 3
入口
0
1
2
3
出口

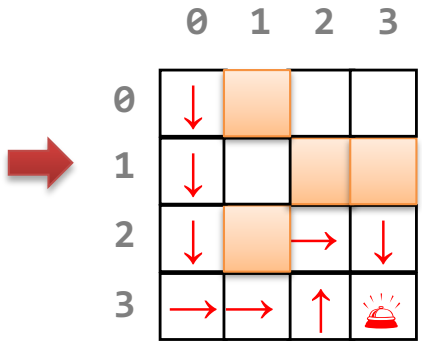

(1)
[0,0]
(2)
[1,0]
pre
(3)
[1,1]×
(4)
[2,0]
(5)
[3,0]
(6)
[3,1]
(7)
[3,2]
(8)
[3,3]


0 1 2 3
0
1
2
3


0 1 2 3
入口
0
1
2
3
出口


[0,0]
di=2
[1,0]
di=1
di=2
[1,1]:×
[2,0]
di=2
[3,0]
di=1
[3,1]
di=1
[3,2]
di=0
[2,2]
di=1
[2,3]
di=2
[3,3]


方位0
方位3
(i,j)
方位1
方位2


0 1 2 3
0
1
2
3

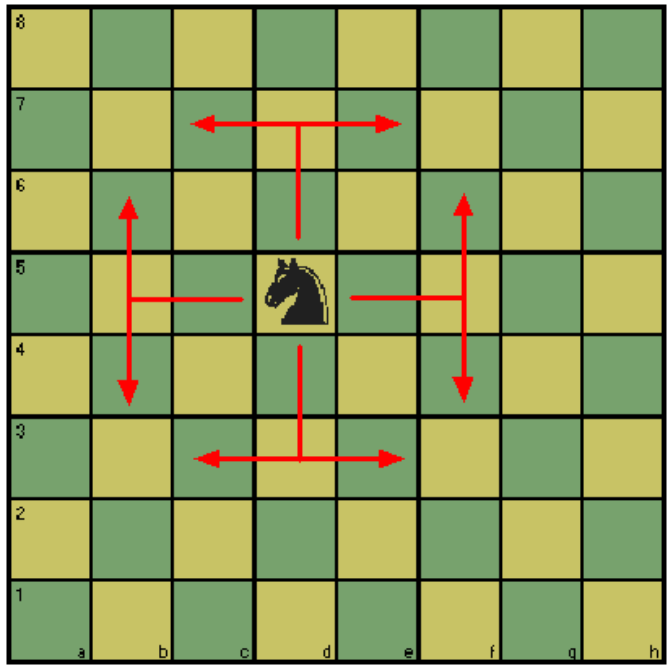


为什么用队列找到的路径一定是最短路径？

骑士最短路径

国际象棋中的骑士，类似于中国象棋中的马，有八种走法(如下图所示)：

1. 左走一格向前走两格；
2. 右走一格向前走两格;
3. 左走一格向后走两格；
4. 右走一格向后走两格
5. 左走两格向前走一格；
6. 左走两格向后走一格
7. 右走两格向前走一格
8. 右走两格向后走一格



给定8*8的棋盘，用字母a~h表示列，数字1~8表示行。棋盘上任意一个格子的位置可由一个字母和一个数字组成的字符串表示。例如，图示中骑士所在位置为d4，它一步可达的8个位置分别为c2, e2,c6, e6, b3, b5, f3, f5。你的任务是写一个程序，对于棋盘上任一两个位置x和y，计算骑士从位置x动到位置y最少需要移动多少步。

车厢重排

一列挂有n节车厢的货运列车途径n个车站，计划在行车途中将各节车厢停放在不同的车站。假设n个车站的编号从1到n，货运列车按照从第n站到第1站的顺序经过这些车站，且将与车站编号相同的车厢卸下。

货运列车的各节车厢以随机顺序入轨，为方便列车在各个车站卸掉相应的车厢，则须重排这些车厢，使得各车厢**从前往后**依次编号为1到n，这样在每个车站只需**卸掉当前最后一节车厢**即可。

车厢重排可通过**转轨站**完成。一个转轨站包含1个入轨（I），1个出轨（O）和 k 个位于入轨和出轨之间缓冲轨（H_i）。请设计合适的算法来实现火车车厢的重排。

车厢重排

本质：将一个无序序列转换成一个以队列方式组织的有序序列

- 转换过程采用缓冲轨存储尚未确定输出次序的车厢，满足递增或增减的特性即可，以栈或队列组织均可。

若将每个缓冲轨看成一个队列

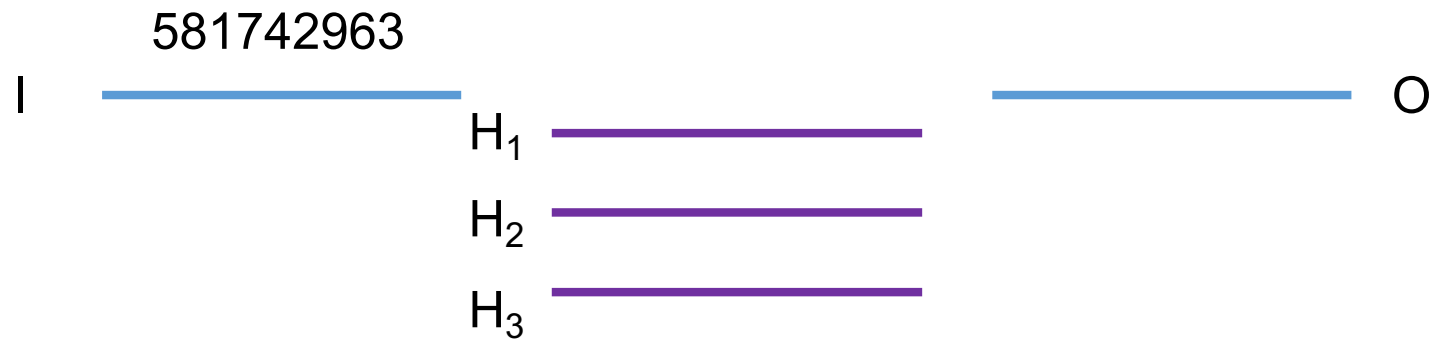
- 转化为：将一个长度为 n 的随机序列（车厢进入入轨），通过 k 个缓冲队列，输出到一个队列（出轨）
- 重排规则：

1. 一个车厢从 入轨 移至 出轨 或 缓冲轨；
2. 一个车厢 只有在其编号恰是下一个待输出编号时，可移到出轨；
3. 一个车厢移到某个缓冲轨，仅当其编号大于该缓冲轨中队尾车厢的编号，若多个缓冲轨满足这一条件，则选择队尾车厢编号最大的，否则选择一个空缓冲轨，若无空缓冲轨则无法重排。

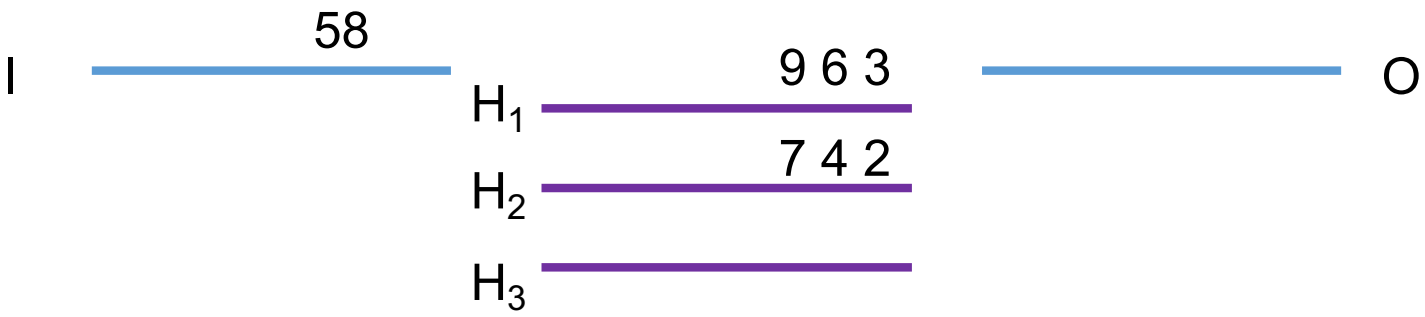
车厢重排

示例：9节车厢的货车，使用3个容量为3的缓冲轨

初始状态：9节车厢均在入轨排队

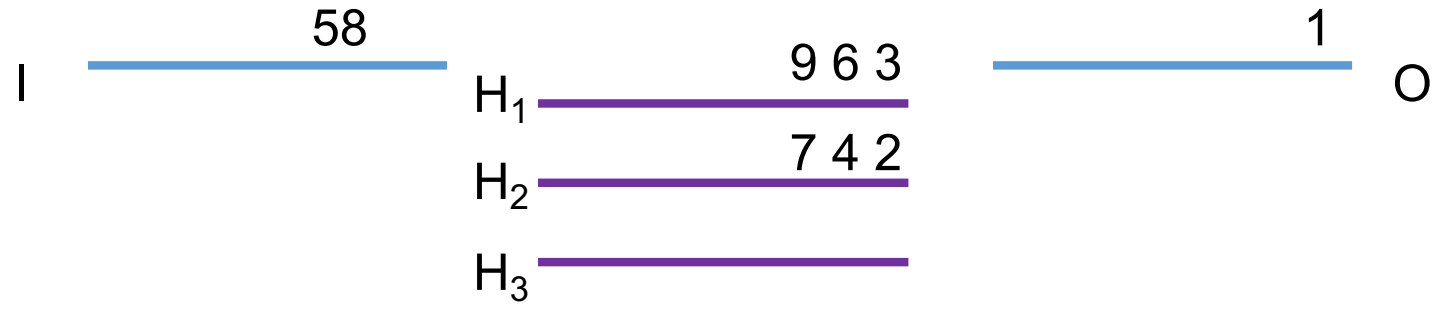


入轨的车厢按序分别进入缓冲轨　H1、H2、H3



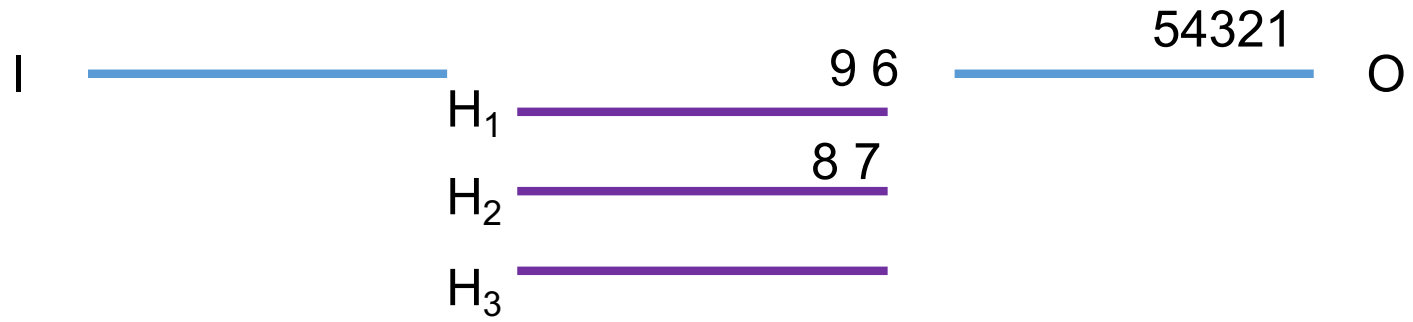
车厢重排

车厢1从H3出队进入出轨



车厢2、3、4按序分别从H2、H1、H2出队进入出轨；

车厢8进入缓冲轨H2 ，车厢5直接进入出轨



车厢重排

各缓冲轨中车厢按编号进入出轨



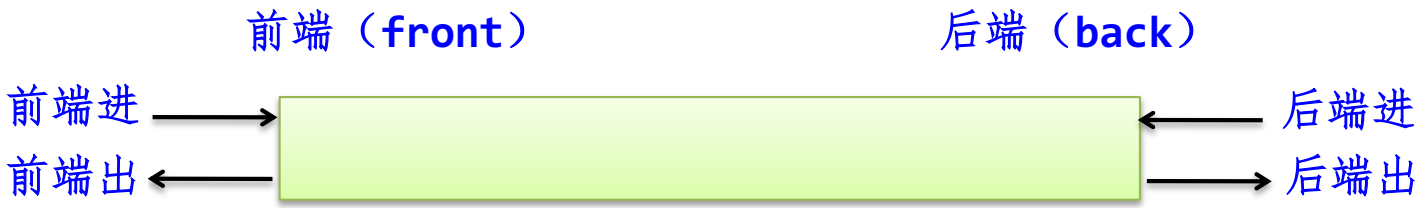
```
bool rearrangeCarsQueue(const vector<int>& cars, int k, vector<int>& res){
    vector<queue<int>> buffers; // k个缓冲队列
    int target = 1; //下一个需要输出的车厢编号
    int n = cars.size();
    //遍历入轨的每一节车厢
    for (int c : cars) {
      //情况1：直接输出到出轨
      if (c == target) {
         res.push_back(c);
         target++;
         //循环检查所有缓冲队列的队头，连续输出匹配的车厢
         bool found;
         do {
            found = false;
            for (auto& q : buffers) {
               if (!q.empty() && q.front() == target) {
                  res.push_back(q.front());
                  q.pop();
                  target++;
                  found = true;
                  break; //找到后重新遍历所有队列
               }
            }
         } while (found);
      }
```

```
    else {// 情况2：放入缓冲队列
        int best_idx = -1;
        int max_tail = -1; // 记录满足条件的最大队尾值
        // 寻找所有 队尾 < 当前车厢 的缓冲队列
        for (int i = 0; i < buffers.size(); ++i) {
          if (!buffers[i].empty() && buffers[i].back() < c) {
             if (buffers[i].back() > max_tail) {
                max_tail = buffers[i].back();
                best_idx = i;
             }
          }
        }
        if (best_idx != -1) { // 子情况1：找到最优队列（队尾最大）入队
          buffers[best_idx].push(c);
        } else {// 子情况2：无满足队列，寻找空缓冲队列
           if (buffers.size() < k) {
              queue<int> new_q;
              new_q.push(c);
              buffers.push_back(new_q);
           } else {// 子情况3：无空队列，重排失败
              return false;
           }
        }
      }
   }
   return target == n + 1; // 最终验证：所有车厢都成功输出
}
```

3.2.8 STL中双端队列和优先队列

1. 双端队列

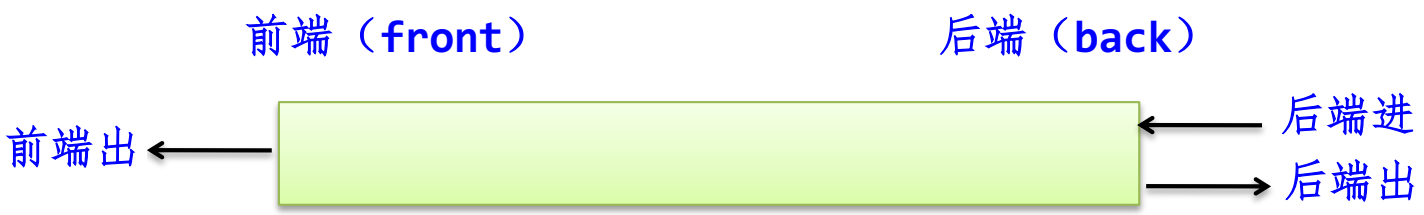
- 双端队列是在队列基础上扩展而来的，其示意图如下图所示。
- 双端队列与队列一样，元素的逻辑关系也是线性关系，但队列只能在一端进队，另外一端出队，而双端队列可以在两端进行进队和出队操作，具有队列和栈的特性，因此使用更加灵活。



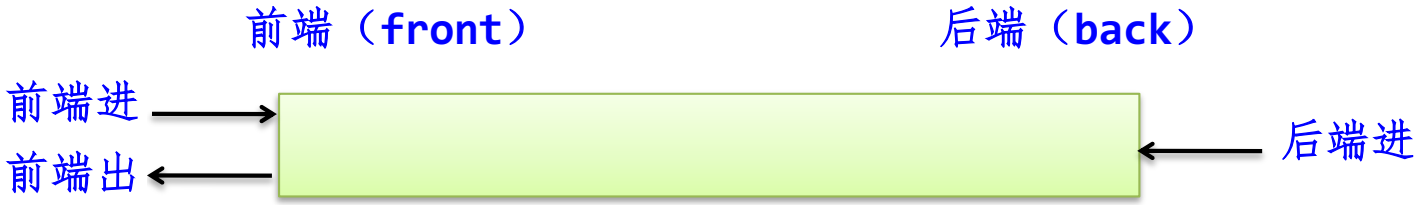
STL中双端队列容器是deque

其他形式的双端队列

- 输入受限的双端队列



- 输出受限的双端队列



定义deque双端队列容器的几种方式

```
deque<int> dq1;                    //定义元素为int的双端队列dq1
deque<int> dq2(10);                //指定dq2的初始大小为10个int元素
deque<double> dq3(10, 1.23);       //指定dq3的10个初始元素的初值为1.23
deque<int> dq4(dq2.begin(), dq2.end()); //用dq2的所有元素初始化dq4
```

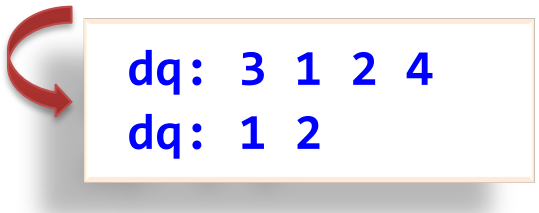
deque主要的成员函数及其说明

成员函数	说明
empty()	判断双端队列容器是否为空队
size()	返回双端队列容器中元素个数
front()	返回队头元素
back()	返回队尾元素
push_front(e)	在队头插入元素e
push_back(e)	在队尾插入元素e
pop_front()	删除队头元素
pop_back()	删除队尾元素
erase()	从双端队列容器中删除一个或几个元素
clear()	删除双端队列容器中所有元素

迭代器成员函数

成员函数	说明
begin()	该函数两个版本返回iterator或const_iterator，引用容器首元素
end()	该函数两个版本返回iterator或const_iterator，引用容器尾元素的后一个位置
rbegin()	该函数两个版本返回reverse_iterator或const_reverse_iterator，引用容器尾元素
rend()	该函数两个版本返回reverse_iterator或const_reverse_iterator，引用容器首元素的前一个位置

```
#include<iostream>
#include <deque>
using namespace std;
int disp(deque<int>& dq)                    //输出dq的所有元素
{  deque<int>::iterator iter;               //定义迭代器iter
   for (iter=dq.begin();iter!=dq.end();iter++)
      printf("%d ",*iter);
   printf("\n");
}
int main()
{  deque<int> dq;                           //建立一个双端队列dq
   dq.push_front(1);                        //队头插入1
   dq.push_back(2);                         //队尾插入2
   dq.push_front(3);                        //队头插入3
   dq.push_back(4);                         //队尾插入4
   printf("dq: "); disp(dq);
   dq.pop_front();                          //删除队头元素
   dq.pop_back();                           //删除队尾元素
   printf("dq: "); disp(dq);
   return 0;
}
```



用双端队列实现栈

- 以前端作为栈底（前端保持不动），后端作为栈顶（后端动态变化），使用成员函数push_back()，pop_back()和back()。
- 以后端作为栈底（后端保持不动），前端作为栈顶（前端动态变化，使用成员函数push_front()，pop_front()和front()。

实际上链表容器list也可以这样作为栈

用双端队列实现普通队列

- 以前端作为队头，后端作为队尾，使用成员函数push_back()，pop_front()和front()。
- 以后端作为队头，前端作为队尾，使用成员函数push_front()，pop_back()和back()。

实际上链表容器list也可以这样作为普通队列

2．优先队列

- 优先队列就是指定队列中元素的优先级，按优先级越大越优先出队，而普通队列中按进队的先后顺序出队，可以看成进队越早越优先。
- 优先队列按照根的大小分为大根堆和小根堆，**大根堆**的元素越大越优先出队（即元素越大优先级也越大），**小根堆**的元素越小越优先出队（即元素越小优先级也越大）。

- STL中的优先队列是priority_queue容器，它和stack/queue一样，它也是一种适配器容器，其底层容器必须是用数组实现的，可以是vector（默认）或者deque，不能是list。
- priority_queue对象的一般定义格式如下：

```
priority_queue<type, container, functional>
```

底层容器，默认为vector

比较函数

优先队列主要的成员函数及其说明

成员函数	说明
empty()	判断优先队列容器是否为空
size()	返回优先队列容器中实际元素个数
push(e)	元素e进队
top()	获取队头元素
pop()	元素出队

下面按type参数的数据类型分为两种情况讨论priority_queue容器的使用。

1）type为内置数据类型

- 对于C/C++内置数据类型，默认的functional是less<T>（小于比较函数）即建立的是大根堆（即元素值越大越优先出队）。
- 可以改为以greater<T>（大于比较函数），这样元素越小优先级的越高（称为小根堆）。

建立大根堆

```
priority_queue<int> big_heap;
                              //默认方式
priority_queue<int,vector<int>,less<int> > big_heap2;
                                    //使用less<T>比较函数
```

建立小根堆

```
priority_queue<int,vector<int>,greater<int> > small_heap;
                              //使用greater<T>比较函数
```

2）type为自定义数据类型

```
class Stud                          //定义类Stud
{
public:
    int no;                         //学号
    string name;                    //姓名
};
```

方式1：在定义类或者结构体类型中重载<运算符（operator <），以指定元素比较方式，如priority_queue<Stud> pq1语句会调用默认的<运算符创建堆pq1（是大根堆还是小根堆由<重载函数体确定）。

方式2：在定义类或者结构体中重载>运算符（operator >），以指定元素比较方式，如priority_queue<Stud，vector<Stud>，greater<Stud> > pq2语句会调用重载>运算符创建堆pq2，此时需要指定优先队列的低层容器（这里为vector，也可以是deque）。

方式3：在单独定义的类或者结构体中重载函数调用运算符()（operator ()），以指定元素比较方式，如priority_queue<Stud,vector<Stud>,StudCmp> pq3语句会调用StudCmp的()运算符创建堆pq3，此时需要指定优先队列的低层容器（这里为vector，也可以是deque）。

```
#include <iostream>
#include <queue>
#include <string>
using namespace std;
class Stud                                  //定义类Stud
{
public:
    int no;                                 //学号
    string name;                            //姓名
    Stud(int n,string na)                   //构造函数
    {   no=n;
        name=na;
    }
    bool operator<(const Stud& s) const       //重载<比较函数
    {                                       //将当前对象跟对象s进行比较
        return no<s.no;                     //按no越大越优先出队
    }
    bool operator>(const Stud& s) const       //重载>比较函数
    {                                       //将当前对象跟对象s进行比较
        return no>s.no;                     //按no越小越优先出队
    }
};
```

```
//类的比较函数,改写operator()
class StudCmp                                   //含重载()成员函数的类
{
public:
   bool operator()(const Stud& s,const Stud& t) const
   {                                            //将对象s跟对象t进行比较
       return s.name<t.name;                    //按name越大越优先出队
   }
};
```

```
int main()
{  Stud a[]={Stud(2,"Mary"),Stud(1,"John"),Stud(5,"Smith")};
   int n=sizeof(a)/sizeof(a[0]);
   //*********************************************
   //方式1：使用Stud类的<比较函数定义大根堆pq1
   //*********************************************
   priority_queue<Stud> pq1(a,a+n);
   cout << "pq1出队顺序: ";
   while (!pq1.empty())                          //按no递减输出
   {  cout << "[" << pq1.top().no << "," << pq1.top().name << "]\t";
      pq1.pop();
   }
   cout << endl;
```



pq1出队顺序: [5,Smith] [2,Mary] [1,John]

```
//*****************************************
//方式2：使用Stud类的>比较函数定义小根堆pq2
//*****************************************
priority_queue<Stud,deque<Stud>,greater<Stud> > pq2(a,a+n);
cout << "pq2出队顺序: ";
while (!pq2.empty())                    //按no递增输出
{  cout << "[" << pq2.top().no << "," << pq2.top().name << "]\t";
   pq2.pop();
}
cout << endl;
```



pq2出队顺序: [1,John] [2,Mary] [5,Smith]

```
    //*************************************************
    //方式3：使用类StudCmp的比较函数定义大根堆pq3
    //*************************************************
    priority_queue<Stud,deque<Stud>,StudCmp> pq3(a,a+n);
    cout << "pq3出队顺序: ";
    while (!pq3.empty())                          //按name递减输出
    {  cout << "[" << pq3.top().no << "," << pq3.top().name << "]\t";
       pq3.pop();
    }
    cout << endl;
    return 0;
}
```



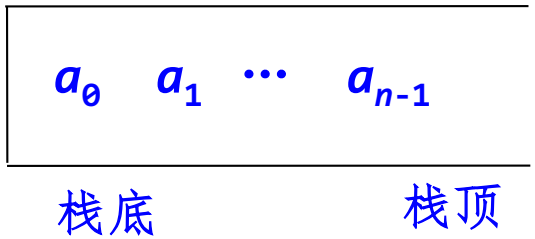
pq3出队顺序: [5,Smith] [2,Mary] [1,John]

3.3* 栈和队列扩展–单调栈和单调队列

3.3.1 单调栈

- 若应用栈时始终维护从栈底到栈顶的元素是有序的，称这样的栈为单调栈。
- 单调栈中元素是有序的，分为单调递增栈和单调递减栈，
- 前者从栈底→栈顶的元素是递增的，

  后者从栈底→栈顶的元素是递减的



$a_0<a_1<\cdots<a_{n-1}$ ⇨ 单调递增栈

$a_0>a_1>\cdots>a_{n-1}$ ⇨ 单调递减栈

- 单调栈的简单应用是由输入序列产生一个单调栈。
- 以单调递减栈为例，只有比栈顶元素小的元素才能直接进栈，否则需要先将栈中比当前元素小的元素出栈，再将当前元素入栈。
- 这样就保证了：栈中保留的都是比当前入栈元素大的值，并且从栈底到栈顶的元素值是单调递减的。
- 适用于涉及到序列中元素大小的问题的求解

序列8, 2, 7, 3, 10

栈为空，8 入栈



2< 8，直接 入栈



7> 2，弹出2，此时7 < 8，入栈



3 < 7，直接入栈



10> 3 及栈中所有元素，依此出栈后，10入栈



【例3.14】一个栈st中有若干个整数，设计一个算法利用一个辅助栈将该栈改变为单调递减栈。



- 设辅助栈为st1，第一步是将st中所有整数进入到st1并使st1为一个单调递增栈，第二步是依次将st1全部整数出栈并进栈st，则st栈改变为单调递减栈。
- 实现第一步操作时，当st栈不空时循环：st栈出栈元素e，将st1栈顶大于等于e的元素退栈并进入st栈中，再将元素e进st1栈。

```
void Sortst(stack<int>& st)
{  stack<int> st1;                          //定义临时栈st1
   int e;

   while (!st.empty())                      //将st1变为递增栈
   {  e=st.top();                           //st出栈元素e
      st.pop();
      while (!st1.empty() && e<st1.top())//e相对st1是反序时
      {  st.push(st1.top());                //退栈st1中大于e的元素并进栈st
         st1.pop();
      }
      st1.push(e);                          //将e进st1栈
   }

   while (!st1.empty())                     //将st1所有元素退栈并进到st栈中
   {  e=st1.top();st1.pop();
      st.push(e);
   }
}
```

单调栈的常见应用问题

- 查找左、右区间中第1个比当前元素大或小的元素
- 确定这个元素是否是区间最值
- 求最大区间
- 求柱状图中的最大矩形

【例3.15】在一个序列中查找当前元素左侧第一个比它大的元素。

考虑一个包含n个元素的无序不重复序列$a_0,a_1,\ldots,a_{n-1}$，如何高效找出每个元素左侧第一个比当前元素大的元素？

用函数$f(i)$表示序列第i个元素a_i之前第一个比它大的元素，

即$f(i)=\max\limits_{0\le j<i,a_j>a_i}\{j\}$，若不存在比$a_i$大的元素，则$f(i)=-1$。

试求出所有元素对应的函数值。

- 示例：以序列（3,4,2,7,5）为例。
- 输出序列为（-1,-1,4,-1,7）



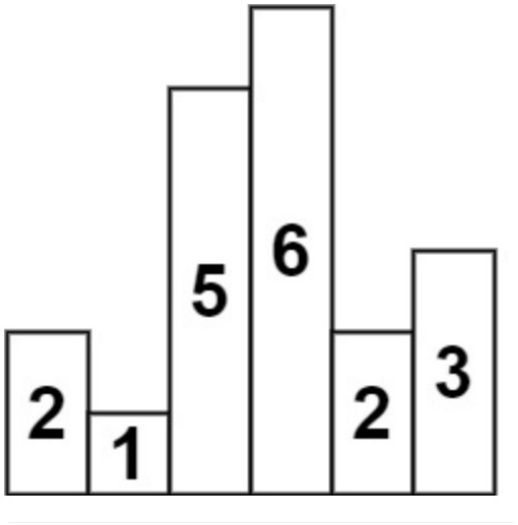
- 对每个元素从后往前去遍历，找到第一个比它大的元素。复杂度$O(n^2)$。
- 基于前述单调递减栈的特性，元素a_i左侧第一个比它大的元素就是入栈时的栈顶元素。
- 从左到右遍历$a_0, a_1, \ldots, a_{n-1}$，构造一个单调递减栈作为中间数据结构，处理到$a_i$时，
  - 若栈为空，则说明左侧不存在比当前元素大的元素，输出-1；
  - 若栈非空，判断栈顶元素是否小于当前元素，将小于当前元素的栈顶元素依次出栈，直到栈为空或栈顶元素大于当前元素，此时输出-1或栈顶元素，再将当前元素入栈。

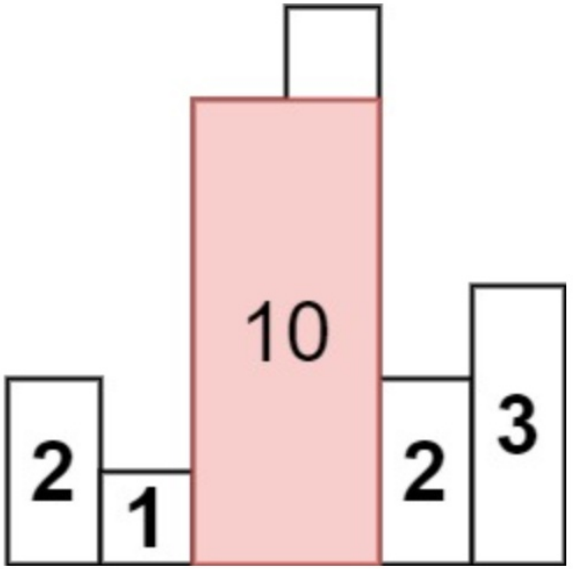
```
// 函数：求解每个元素左侧第一个更大元素
 vector<int> findFirstGreaterLeft(const vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, -1); // 结果数组，初始化为-1
    stack<int> st;          // 单调递减栈
    for (int i = 0; i < n; ++i) {
       // 弹出栈顶所有小于当前元素的值
       while (!st.empty() && st.top() <= nums[i]) {
          st.pop();
       }
       // 栈不为空，栈顶就是第一个更大元素
       if (!st.empty()) {
          res[i] = st.top();
       }
       st.push(nums[i]); // 当前元素入栈
     }
    return res;
 };
```

- 示例：以序列（3,4,2,7,5）为例。
- 输出序列为（-1,-1,4,-1,7）

【例3.15】求柱状图中的最大矩形。【扩展-自学】

一个柱状图是由若干底部对齐、宽度相同但高度不一的矩形排列而成，如左下图所示。考虑包含n个矩形的柱状图，每个矩形的高分别为$a_0,a_1,\ldots,a_{n-1}$，求该柱状图覆盖区域的最大矩形。





【输入】2 1 5 6 2 3

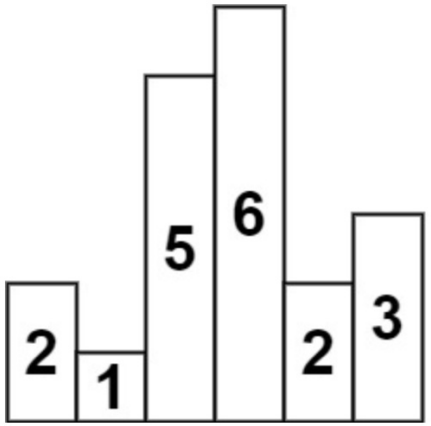
【输出】10



- 解法1：枚举法，对于heights[i..j](j>=i)的j-i+1个柱子，构成的矩形面积为heights[i..j]中最小高度*(j-i+1)。在所有这些矩形面积中比较求最大值。

```
int largestRectangleArea(vector<int>& heights){
        int ans=0;
        for(int j=0;j<heights.size();j++){
            int minh=heights[j];
            for(int i=j;i>=0;i--){
                if(heights[i]<minh)
                  minh=heights[i];
                int s=(j-i+1)*minh;
                if(ans<s) ans=s;
            }

        }
        return ans;
}
```

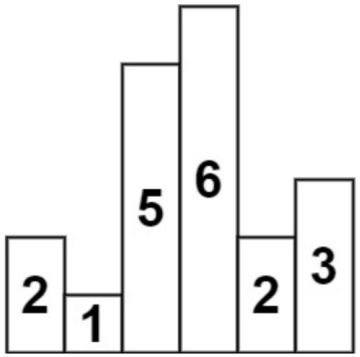




- 解法2：单调栈法。
- 用maxr([i..j])表示[i..j]中最大的矩形面积。如果柱子高度都是递增的，那么maxr([i..i])<…<maxr([i..j-1])< maxr([i..j])。当j+1柱的高度大于j柱高度时，maxr([i..j+1])> maxr([i..j])，没有必要求前面的所有面积。只有当maxr([i..j+1])>maxr([i..j])不成立时再求这些面积从中求出最大即可。
- 栈中存储柱子的下标，每个柱子仅进栈一次，从栈底到栈顶柱子的高度是递增的。从左到右遍历柱子，当栈空或者当前柱子j的高度大于或等于栈顶柱子高度时，将j进栈并j++，即维护栈中柱子高度的递增性，否则对栈中所有高度大于j的柱子求矩形面积S。退栈柱子k，求该柱到j-1位置为止构成的最大矩形面积。其高是柱k的高度，长度分两种情况。



- 情况1：柱k退栈后栈非空，st.top为新栈顶柱子，求以j-1结尾，以k柱为最小高度的矩形S面积， 面积为a_k*(j-k)=a_k*(j-st.top-1)
- 情况2：柱k退栈后栈空，说明k柱在[0..j-1]中高度最小，所以面积为a_k*j。
- 当所有柱子遍历完需要对栈中所有柱子求矩形面积，为此在输入高度末尾添加一个高度为0的虚拟柱子。
- 以{2,1,5,6,2,3,0}为例





- 情况1：柱k退栈后栈非空，st.top为新栈顶柱子，求以j-1结尾，以k柱为最小高度的矩形S面积，按照单调栈的操作又可以分为两种情况处理：

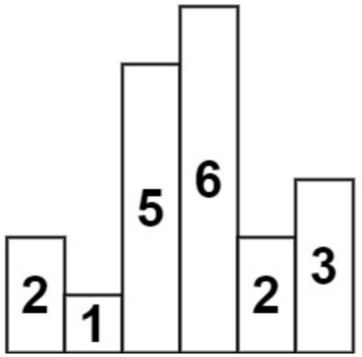
  ① st.top和k相邻，则面积为$a_k*(j-k)=a_k*(j-st.top-1)$

  ② st.top<k-1,即柱子st.top+1到柱子k-1之间的柱子在k出栈前已经出栈了，说明它们的高度都大于柱k，需要计入矩形S中。面积为$a_k*(j-k)=a_k*(j-st.top-1)$。

- 情况2：柱k退栈后栈空，说明k柱在[0..j-1]中高度最小，所以面积为a_k*j。

- 当所有柱子遍历完需要对栈中所有柱子求矩形面积，为此在输入高度末尾添加一个高度为0的虚拟柱子。

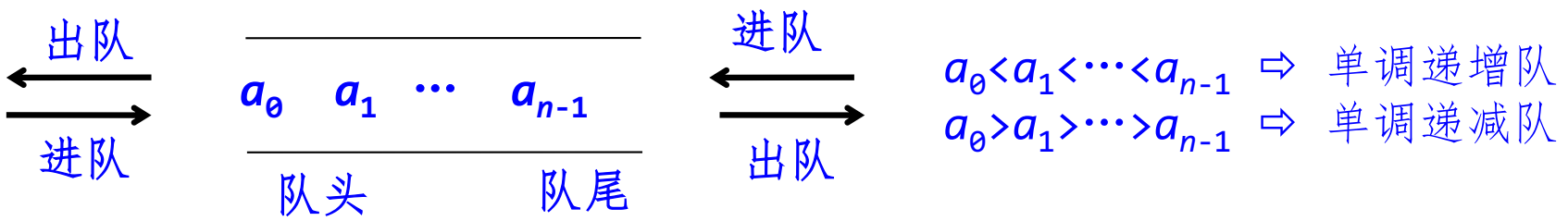
- 以{2,1,5,6,2,3,0}为例



```
int largestRectangleArea(vector<int>& heights){
        heights.push_back(0); //在h末尾添加一个高度为0的虚柱子
        int n=heights.size();
        stack<int> st;
        int ans=0;     //存放最大矩形面积，初始为0
        for(int j=0;j<n;j++){
            if(st.empty() || heights[j]>heights[st.top()])
               st.push(j); //栈空或高度是单调递增的,直接进栈
            else{
               while(!st.empty() && heights[st.top()]>=heights[j]){
                       int tmp=st.top(); st.pop();  //退栈tmp
                       int length;
                       if(st.empty())
                          length=j;
                       else
                           length=j-st.top()-1;
                       //求heights[st.top()+1..j-1]的矩形面积
                       int area=heights[tmp]*length;
                       ans=max(ans,area);   //维护ans为最大矩形面积
               }
               st.push(j);              //最后j进栈
            }
        }
        return ans;
}
```

3.3.2 单调队列

- 若应用队列时始终维护从队头到队尾的元素是有序的，称这样的队列为单调队列。
- 单调队列中元素是有序的，分为单调递增队列和单调递减队列，前者从队头到队尾的元素是递增的，后者从队头到队尾的元素是递减的。



单调队列通常用于维护一个输入序列中指定区间的最值（即指定区间中的最大值或者最小值），假设区间长度为k，其基本操作是队头出队、队尾进队和出队：

① 队中元素顺序（从队头到队尾）是输入序列中元素从前向后的相对顺序。

② 保持队列的单调性，队头为最值，求最大值时采用单调递减队**minqu**，求最小值时采用单调递增队**maxqu**。

③ 元素x进队总是从队尾进，如果是单调递减队，若x≤队尾元素，直接将x进队，否则从队尾出队元素直到x≤队尾元素（称为去尾操作），再将x进队。如果是单调递增队，若x≥队尾元素，直接将x进队，否则从队尾出队元素直到x≥队尾元素，再将x进队。

④ 从队头出队超过区间的元素（过期元素），如果当前位置是i，若i-队头元素序号≥k，则说明队头元素超过当前区间，需要出队（称为掐头操作）。所以单调队列中需要保存元素的时间戳以便确定是否过期。

说明

通常单调队列采用STL中的deque容器来实现，由于输入序列中每个元素只会进队和出队一次，所以单调队列的时间复杂度是$O(n)$，是非常高效的。

单调队列的常见应用问题

- 维护数组最值：在长度为n的序列中，求每个长度为m的区间中的最大最小元素。
- 满足约束的子序列问题，包括下述问题在内的多种应用：
  - 连续子序列问题。给定一个整数K和一个序列，求和至少为K的最短子序列。
  - 绝对差不超过约束的子序列问题。对于一个整数序列A，一个表示限制的整数L，返回最长连续子序列的长度，其中任意两个元素之间的绝对差必须小于或者等于L。
- 跳格子游戏：给定一个元素为格子的序列A，从第一个元素开始往后跳，每次最远跳K步，且不会跳出序列之外，每次跳跃的收益为所跳到格子的值，而最终得分为所有跳到格子值的加和，求跳到最后一个位置时的最大得分。

【例3.16】求滑窗区间最小/最大值。

给定一个具有n个元素的序列$a_0,a_1,\ldots,a_{n-1}$和一个长度为k的窗口，请给出窗口内区间最小/最大值。即，$\forall 0 \le i \le n-k$，求$\min\{a_i,a_{i+1},\ldots,a_{i+k-1}\}$和 $\max\{a_i,a_{i+1},\ldots,a_{i+k-1}\}$

示例：给定n=8的序列{1,3,-1,-3,5,3,6,7}，和长度为3的窗口，其区间最小/最大值如下表所示。

窗口位置	最小值	最大值
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

最小值序列：-1 -3 -3 -3 3 3

最大值序列：3 3 5 5 6 7



- 若窗口区间为[i,j]，则对应的窗口中元素集合为$\{a_i, a_{i+1}, \ldots, a_j\}$，当窗口右移一位时，$a_i$将移出窗口，而$a_{j+1}$进入窗口。集合$\{a_i, a_{i+1}, \ldots, a_j\}$中只要有比$a_{j+1}$大的元素，那么窗口右移后，最小值肯定不是该集合中那些比a_{j+1}大的元素，故可将其丢弃。即，两个位置$i \le j$，若$a_i \ge a_j$，则没有必要存储a_i。因为若位置i为区间最小值，则i在窗口内时，j一定也在窗口内，且a_j更小。

- 从左向右滑动窗口时，维护序列$Q=\{q_0, q_1, \ldots, q_l\}$中不存在任意两个位置i<j满足$q_i \ge q_j$，换言之，Q是一个单调递增的序列。因此，可通过构建一个单调递增队列作为中间数据结构来组织进入窗口的元素，队首即为当前的最小元素。

- 每次窗口右移一位，将新进窗口的元素a_{i+k}入队并检查队列的单调性，删除队列中比a_{i+k}大的元素，同时若队首已滑出窗口则将其删除。由于每个元素最多入队1次、出队1次，故时间复杂度为O(n)。

- 求滑窗区间最值具有广泛的应用，例如，求股票在过去一年内的最高点、求最近半个月的最高气温和最低气温。

```
// 计算滑动窗口最大值（单调递减队列）
vector<int> getMaxWindow(vector<int>& nums, int k) {
    vector<int> res;
    deque<int> dq;
    // 存储数组下标，保证对应数值递减
   for (int i = 0; i < nums.size(); ++i) {
      // 1. 移除队头：超出窗口左边界的元素
      while (!dq.empty() && dq.front() <= i - k)
           dq.pop_front();

      // 2. 移除队尾：比当前元素小的，不可能是最大值
      while (!dq.empty() && nums[dq.back()] < nums[i])
           dq.pop_back();

      // 3. 当前元素下标入队
      dq.push_back(i);

      // 4. 窗口形成，记录最大值
      if (i >= k - 1)
           res.push_back(nums[dq.front()]);
    }
    return res;
}
```